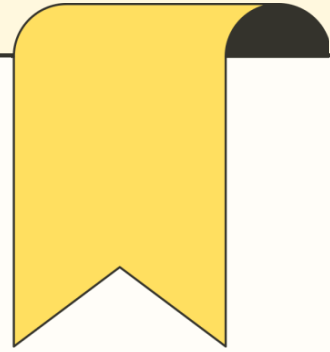
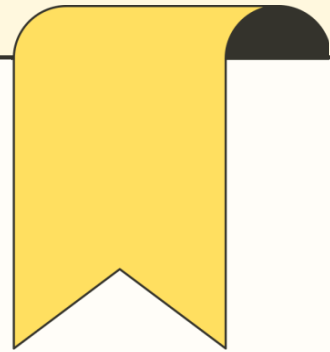




AMBA AXI BUS SPI & I2C 설계 및 UVM 검증



CONTENTS

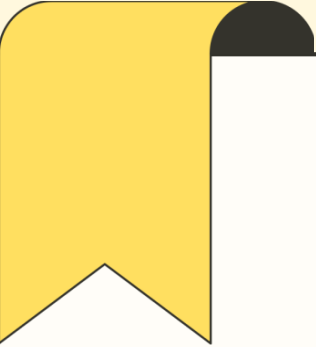
01 Introduction

02 AXI Bus

03 SPI

04 I2C

05 Discussion



Introduction

Project Goal & Environment

Project Goal

1. AXI Bus 구조 설계

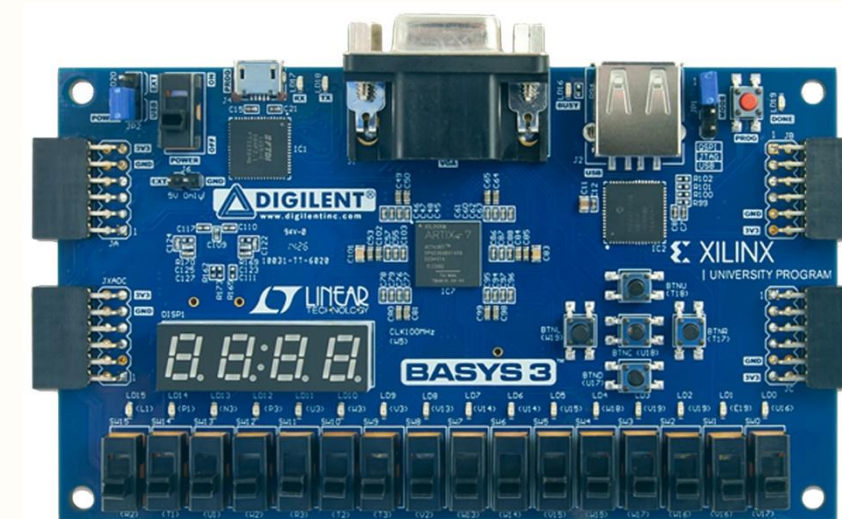
- AXI BUS Protocol에 대한 개념 이해 및 설계
- AXI BUS Protocol을 활용한 Peripheral 제어 설계

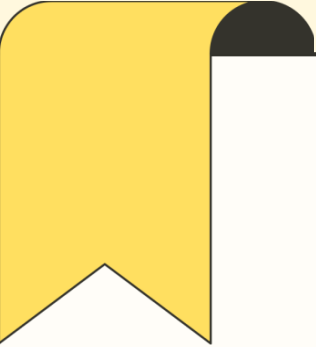
2. SPI Protocol 기반 설계

- SPI Protocol에 대한 개념 이해 및 설계
- SPI Protocol을 활용한 FND 제어
- Synopsys, Verdi를 활용한 SPI Loopback 검증

3. I2C Protocol 기반 설계

- I2C Protocol에 대한 개념 이해 및 설계
- I2C Protocol을 활용한 LED 제어





AXI Bus

AXI Bus

AXI 5가지 독립 채널

1. AW - Write Address

- Master -> Slave
- 쓰기 트랜잭션의 주소와 제어 정보 전송

2. W - Write Data

- Master -> Slave
- 쓰기 데이터를 전송

3. B - Write Response

- Slave -> Master
- 쓰기 트랜잭션의 완료 여부를 알림

4. AR - Read Address

- Master -> Slave
- 읽기 트랜잭션의 주소와 제어 정보를 전송

5. R - Read Data

- Slave -> Master
- 읽기 데이터와 응답 정보를 함께 전송

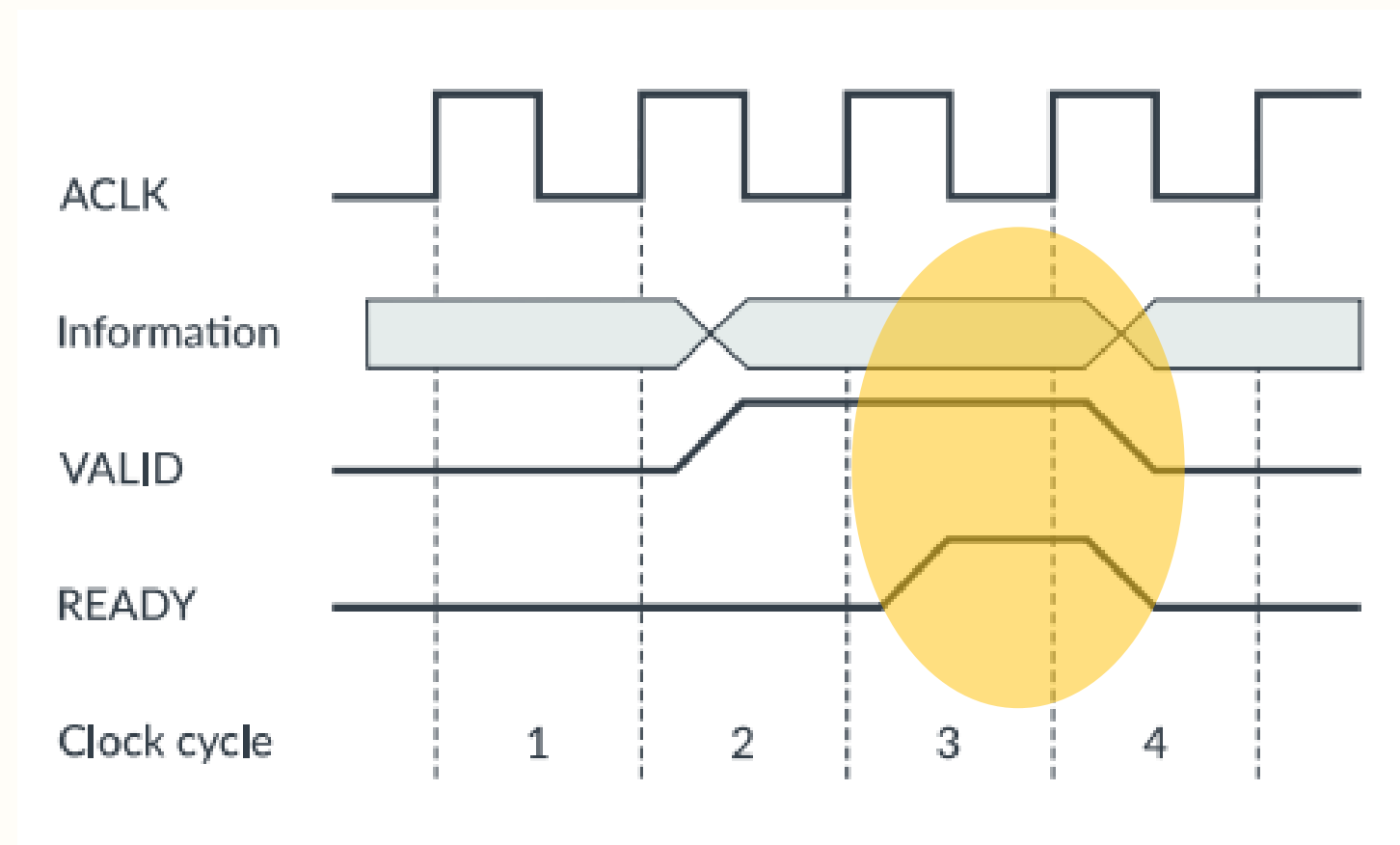
AXI : Advanced eXtensible Interface
: 고성능 SoC 설계를 위한 표준 인터페이스



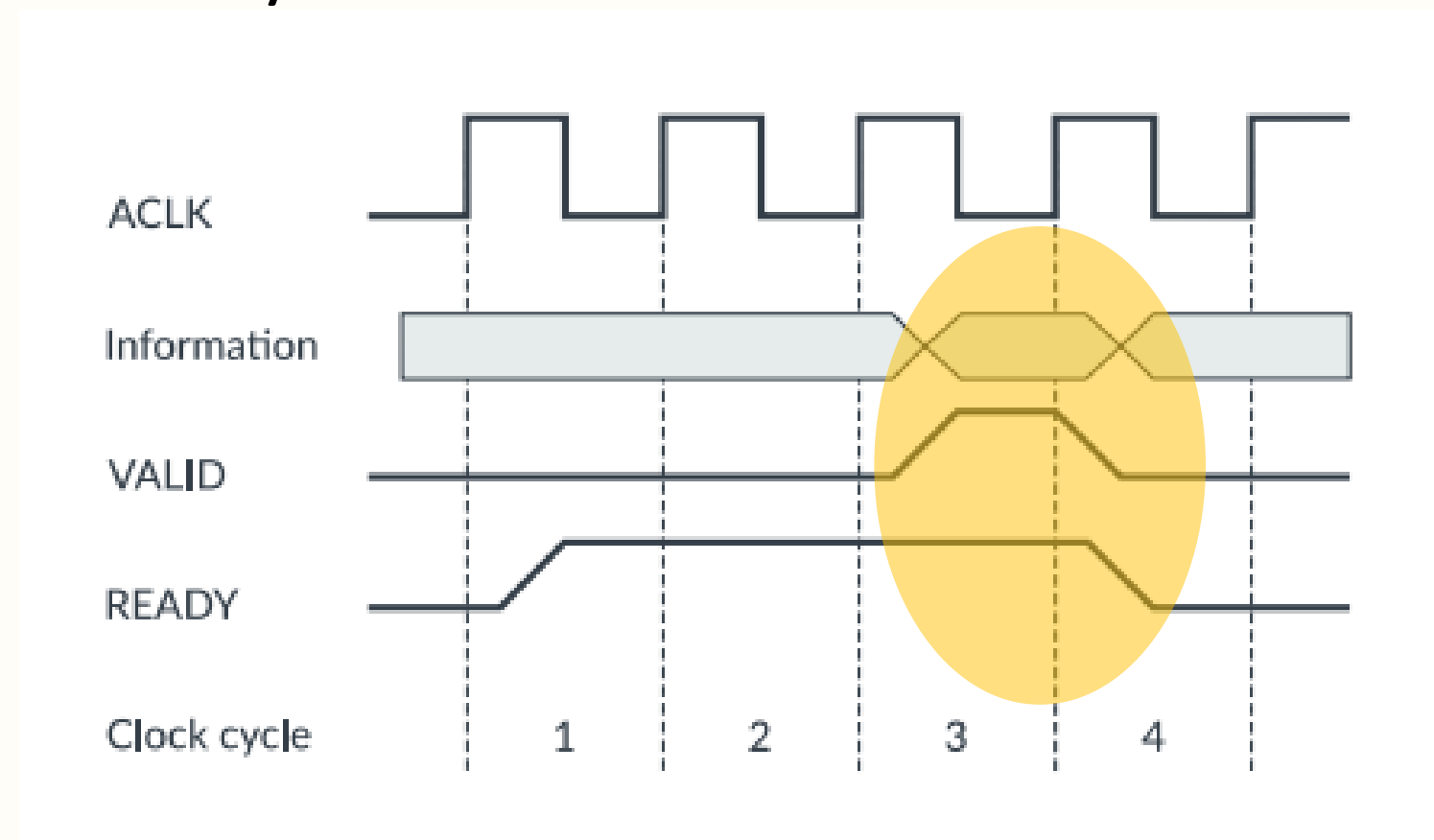
AXI Bus Protocol - VALID/READY Handshake

AXI 통신의 핵심 제어 메커니즘, 모든 채널에서 정보를 안전하게 전송하기 위해 사용

Valid 먼저 발생

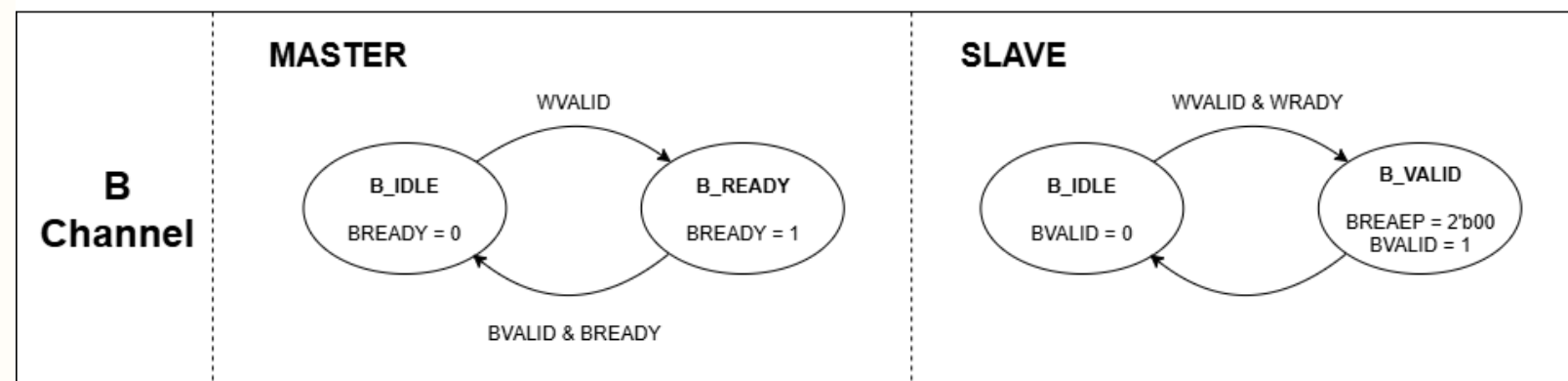
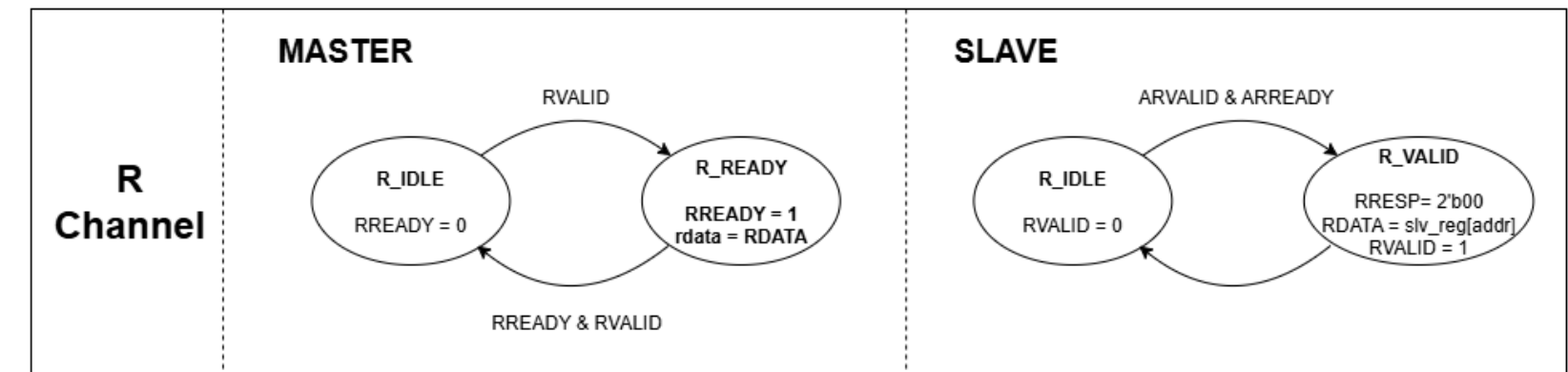
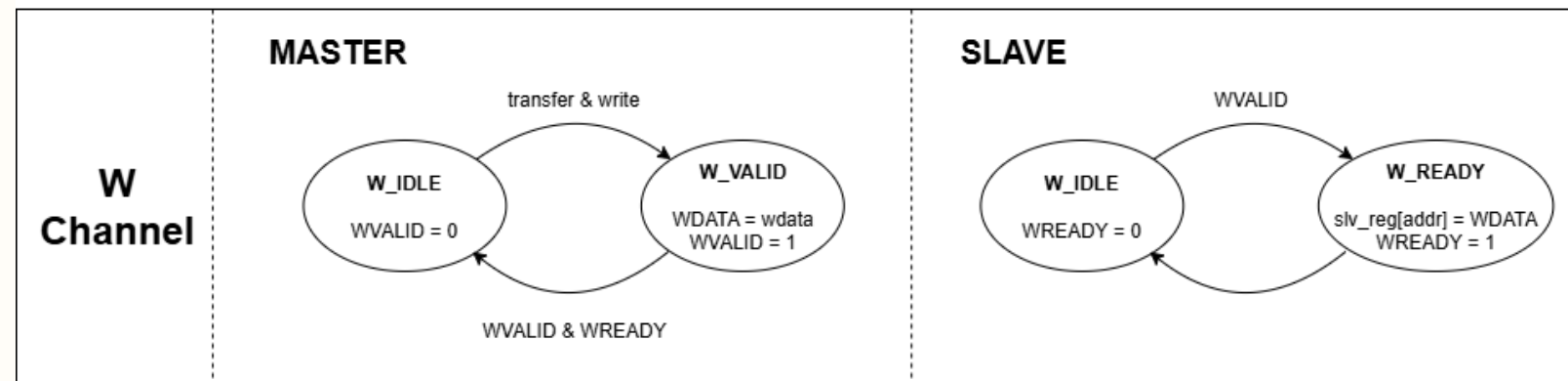
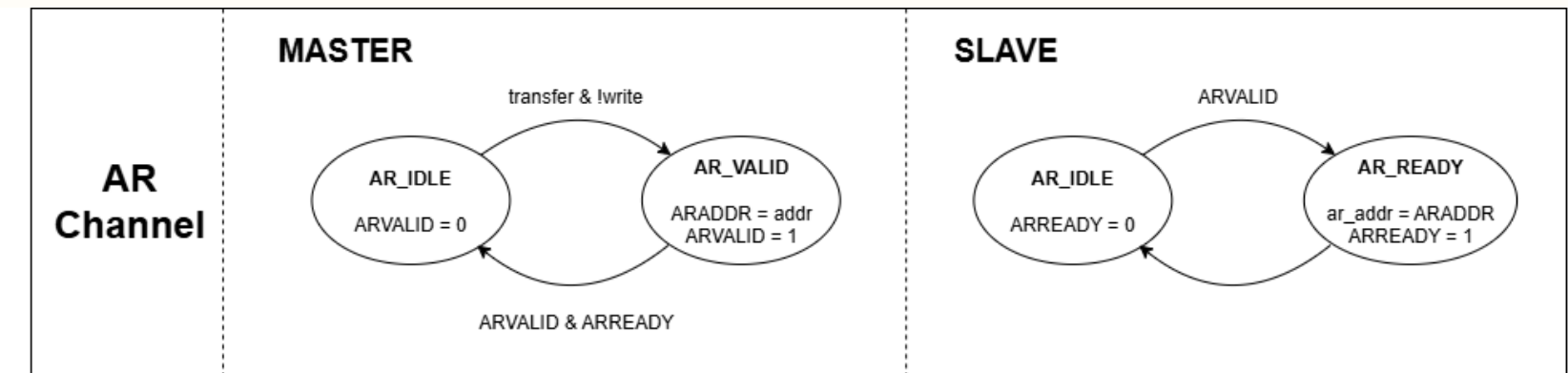
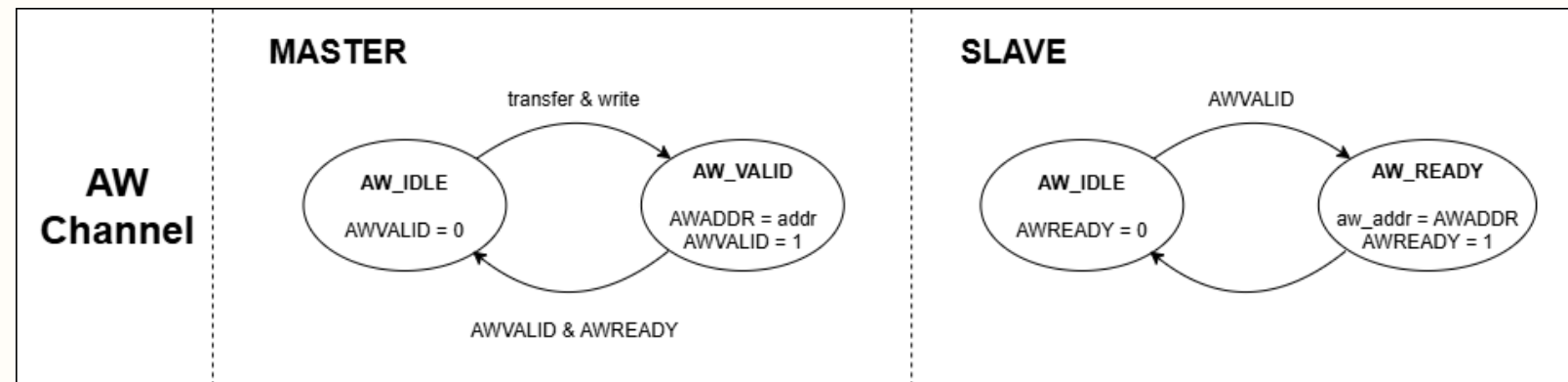


Ready 먼저 발생



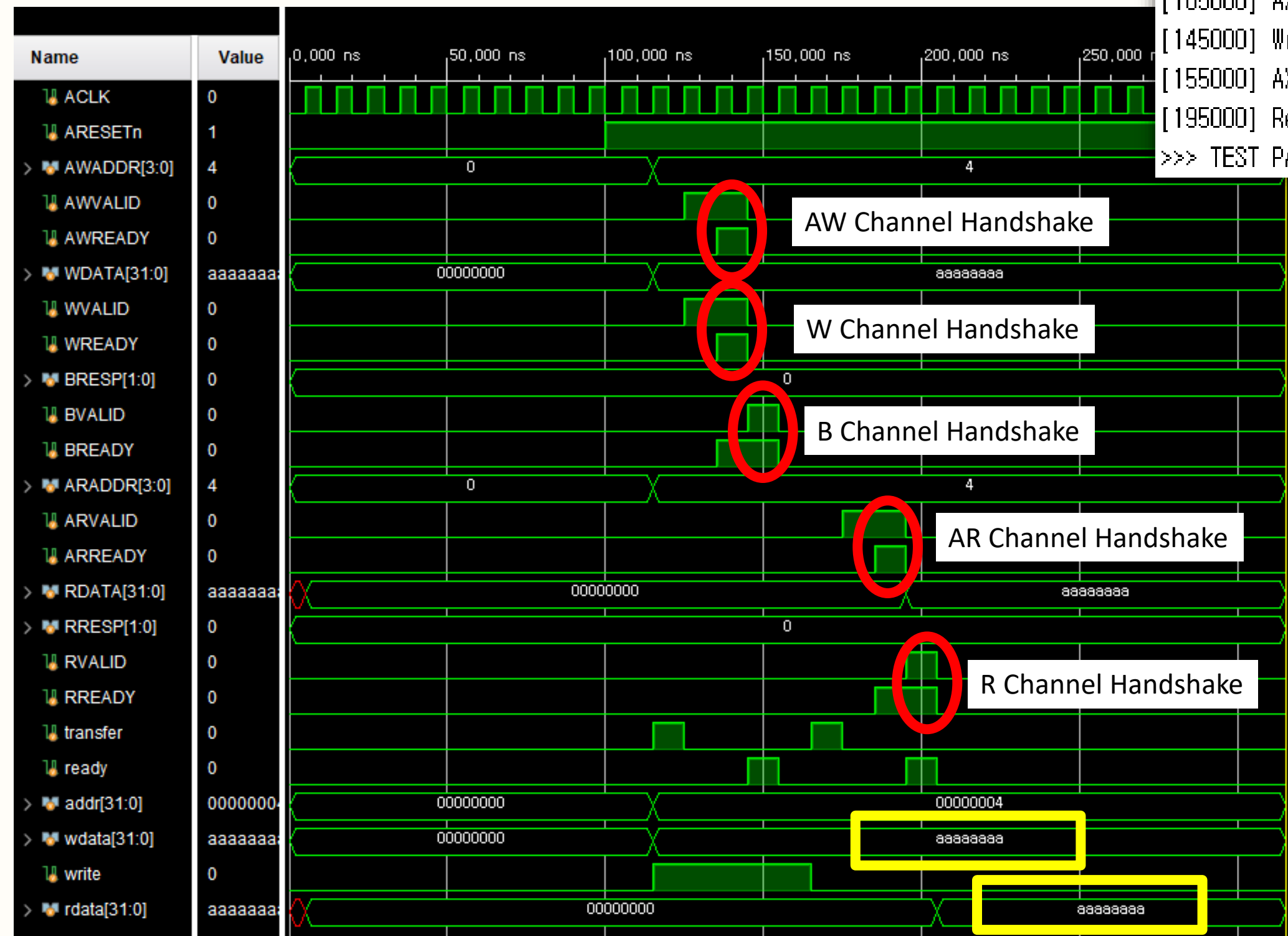
VALID 신호와 READY 신호가 모두 High일 때, 하나의 클럭 사이클 동안 정보 전송 (DATA Write/Read)

AXI Bus Protocol

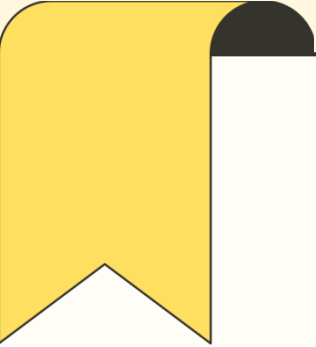


**VALID == 1 && READY == 1
=> Handshake**

AXI Bus Protocol - Simulation

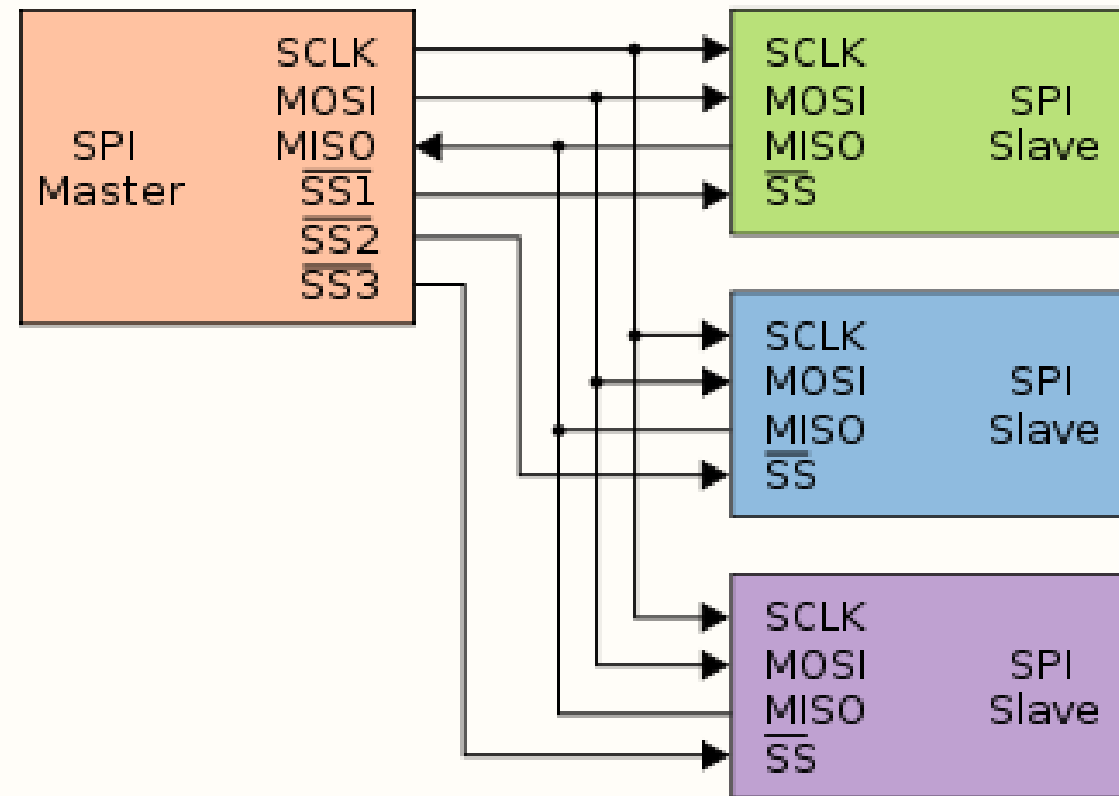


```
[105000] AXI WRITE to 0x4
[145000] Write to 0x4 complete.
[155000] AXI READ from 0x4
[195000] Read from 0x4 complete.
>>> TEST PASS: Read data 0xaaaaaaaa matches 0xAAAAAAAA
```



SPI

SPI



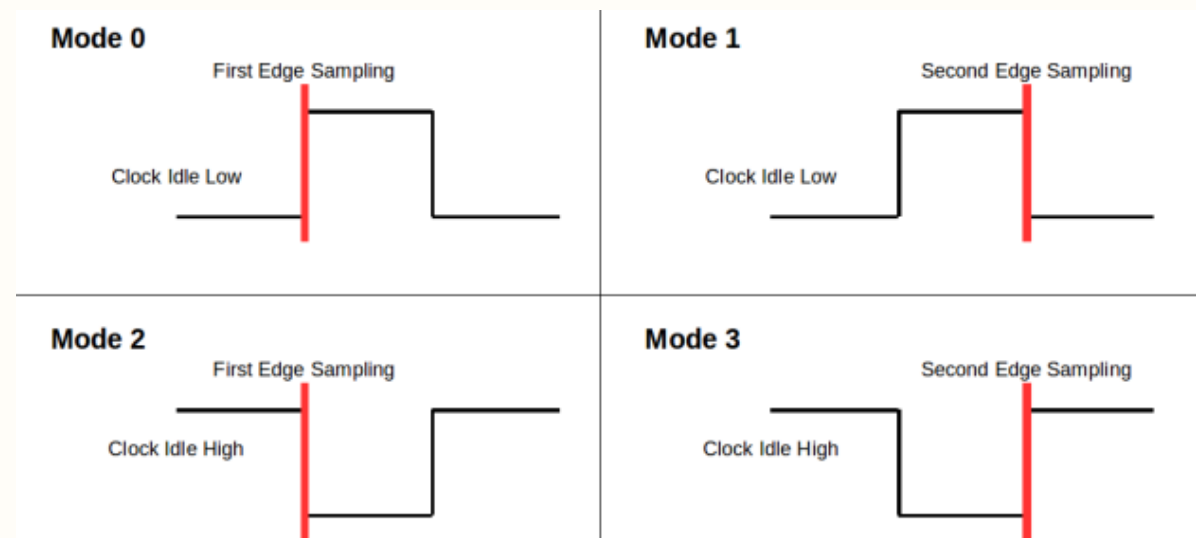
SPI : Serial Peripheral Interface

: 데이터를 고속으로 주고받는 동기 직렬 통신 프로토콜

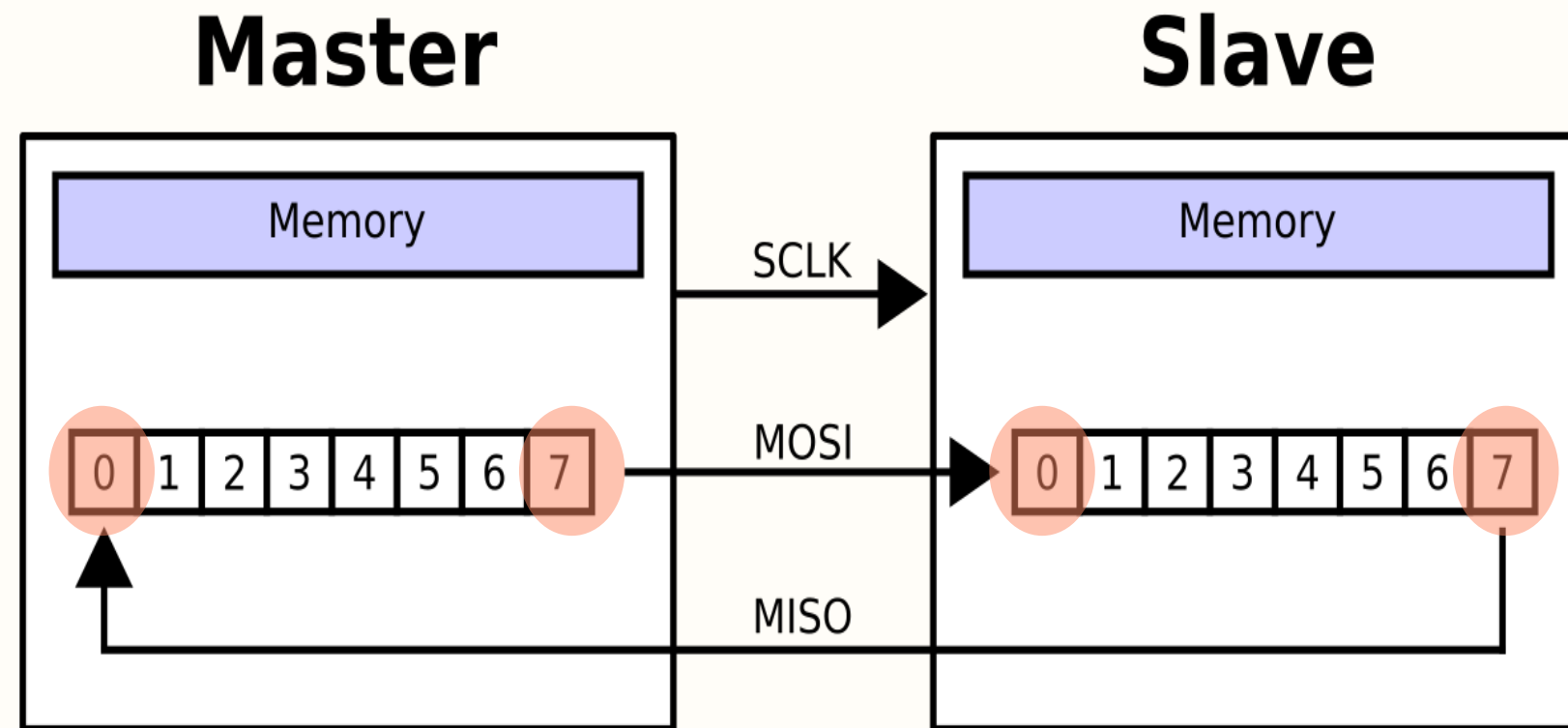
- Master – Slave 간 **1:N 통신 구조**
- 전이중 통신 (Full Duplex) : **데이터를 양방향으로 동시에 송수신**
- **SS(Slave Select / CS)** 신호를 통한 **특정 Slave 선택** (Active Low)
- 매우 **빠른** 속도의 **동기식 통신**
- CPOL, CPHA를 통한 **4가지 동작 Mode 존재**

CPOL (Clock Polarity), CPHA (Clock Phase)

- ✓ CPOL = 0, CPHA = 0 : CLK = LOW, Rising Edge Sampling (Mode 0)
- ✓ CPOL = 0, CPHA = 1 : CLK = LOW, Falling Edge Sampling (Mode 1)
- ✓ CPOL = 1, CPHA = 0 : CLK = HIGH, Falling Edge Sampling (Mode 2)
- ✓ CPOL = 1, CPHA = 1 : CLK = HIGH, Rising Edge Sampling (Mode 3)



SPI Protocol



Master 기준

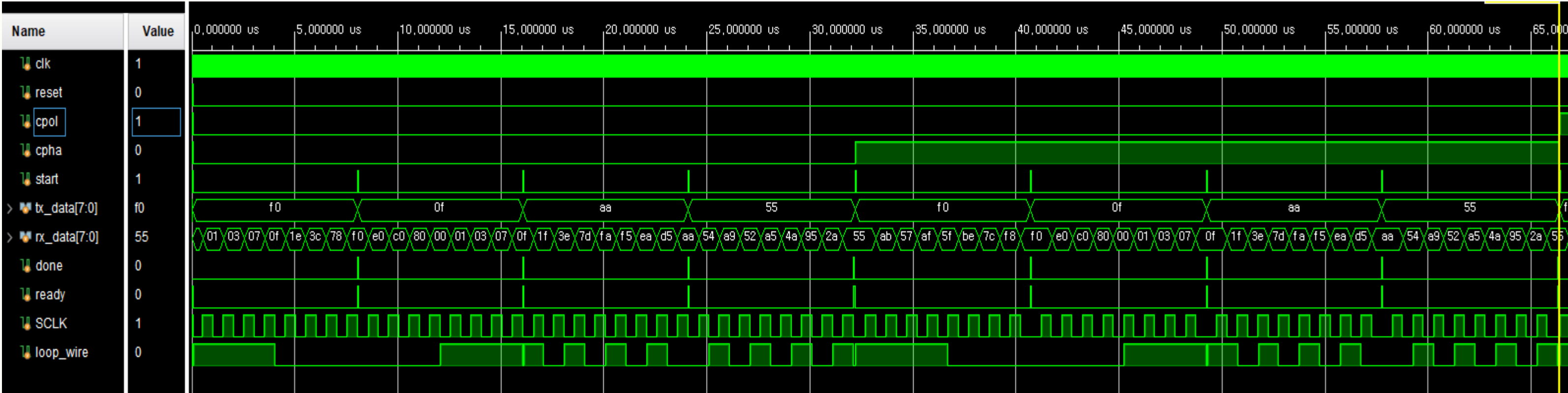
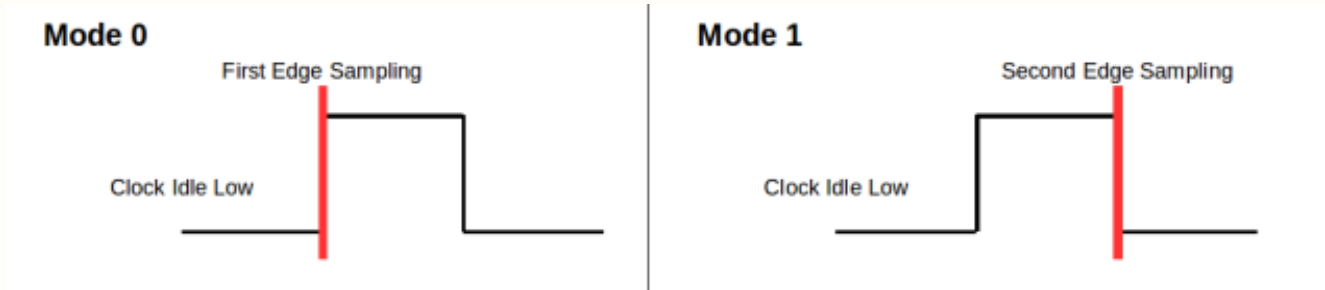
- **Write** : MOSI로 1bit data 8번 전송, 동시에 **MISO** Dummy Data 수신.
- **Read** : MISO로 1bit data 8번 수신, 동시에 **MOSI** Dummy Data 출력.

Slave 기준

- **Write** : MOSI로 1bit data 8번 수신, 동시에 **MISO** Dummy Data 출력.
- **Read** : MISO로 1bit data 8번 전송, 동시에 **MOSI** Dummy Data 수신.

SPI Protocol

– Simulation



Rising Edge Sampling (Mode 0)

```
--- Starting Test for SPI Mode 0 (CPOL=0, CPHA=0) ---
Time=15000 | Mode(P0,A0) | Ready=1, Done=0 | TX=0x00 | RX=0x00 | SCLK=0, Loop=0
Time=25000 | Mode(P0,A0) | Ready=0, Done=0 | TX=0xf0 | RX=0x00 | SCLK=0, Loop=0

Time=31595000 | Mode(P0,A0) | Ready=0, Done=0 | TX=0x55 | RX=0x55 | SCLK=1, Loop=1
Time=32085000 | Mode(P0,A0) | Ready=0, Done=1 | TX=0x55 | RX=0x55 | SCLK=0, Loop=1
--- Mode 0 PASSED: TX(0x55) == RX(0x55) ---
```

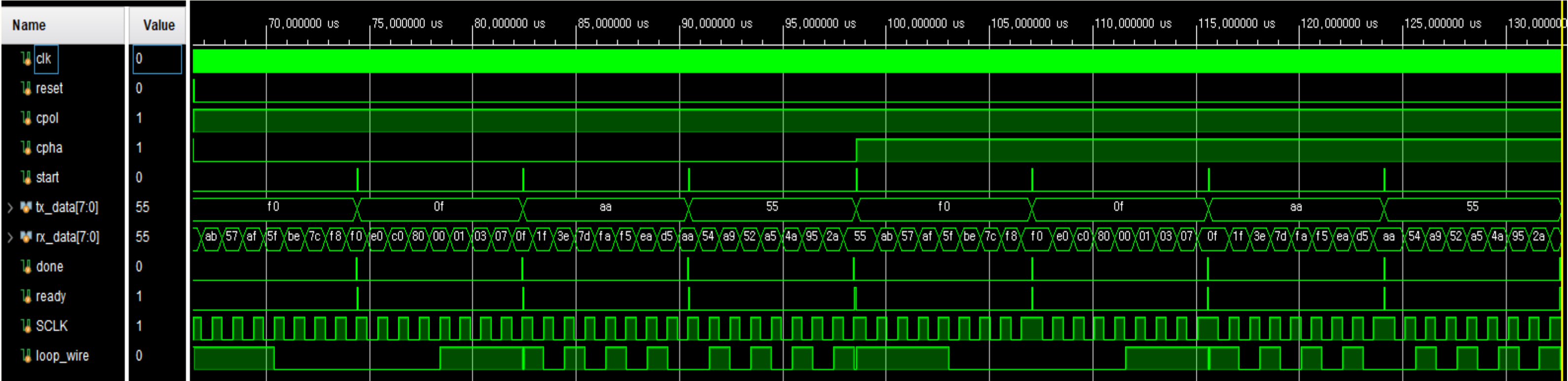
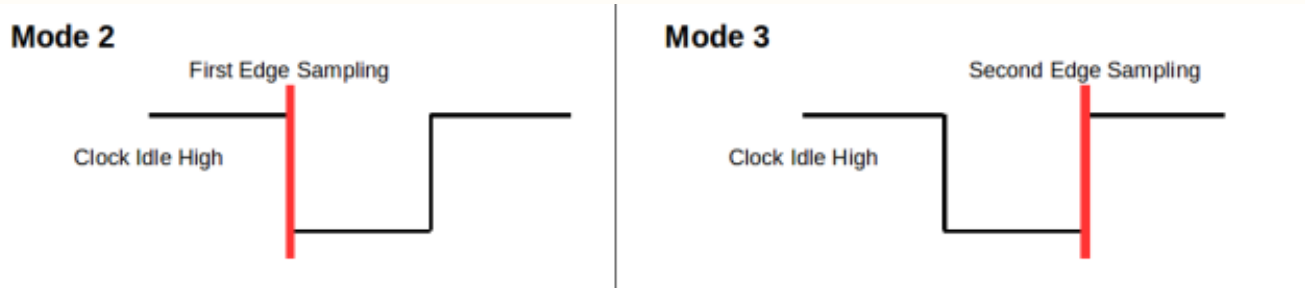
Falling Edge Sampling (Mode 1)

```
--- Starting Test for SPI Mode 1 (CPOL=0, CPHA=1) ---
Time=32195000 | Mode(P0,A1) | Ready=0, Done=0 | TX=0xf0 | RX=0x55 | SCLK=0, Loop=0
Time=32205000 | Mode(P0,A1) | Ready=0, Done=0 | TX=0xf0 | RX=0x55 | SCLK=0, Loop=1

Time=65765000 | Mode(P0,A1) | Ready=0, Done=0 | TX=0x55 | RX=0x55 | SCLK=0, Loop=1
Time=66255000 | Mode(P0,A1) | Ready=0, Done=1 | TX=0x55 | RX=0x55 | SCLK=0, Loop=1
--- Mode 1 PASSED: TX(0x55) == RX(0x55) ---
```


SPI Protocol

– Simulation



Falling Edge Sampling (Mode 2)

```
--- Starting Test for SPI Mode 2 (CPOL=1, CPHA=0) ---  
Time=66365000 | Mode(P1,A0) | Ready=0, Done=0 | TX=0xf0 | RX=0x55 | SCLK=1, Loop=0  
Time=66375000 | Mode(P1,A0) | Ready=0, Done=0 | TX=0xf0 | RX=0x55 | SCLK=1, Loop=1  
  
Time=97935000 | Mode(P1,A0) | Ready=0, Done=0 | TX=0x55 | RX=0x55 | SCLK=0, Loop=1  
Time=98425000 | Mode(P1,A0) | Ready=0, Done=1 | TX=0x55 | RX=0x55 | SCLK=1, Loop=1  
--- Mode 2 PASSED: TX(0x55) == RX(0x55) ---
```

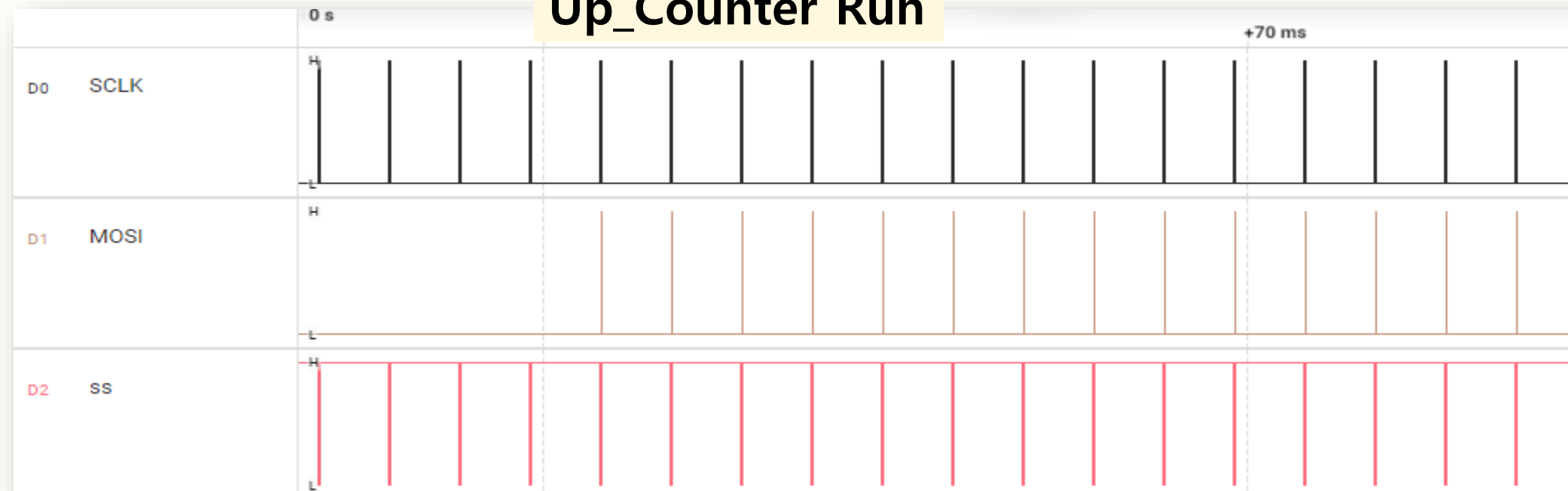
Rising Edge Sampling (Mode 3)

```
--- Starting Test for SPI Mode 3 (CPOL=1, CPHA=1) ---  
Time=98535000 | Mode(P1,A1) | Ready=0, Done=0 | TX=0xf0 | RX=0x55 | SCLK=1, Loop=0  
Time=98545000 | Mode(P1,A1) | Ready=0, Done=0 | TX=0xf0 | RX=0x55 | SCLK=1, Loop=1  
  
Time=132105000 | Mode(P1,A1) | Ready=0, Done=0 | TX=0x55 | RX=0x55 | SCLK=1, Loop=1  
Time=132595000 | Mode(P1,A1) | Ready=0, Done=1 | TX=0x55 | RX=0x55 | SCLK=1, Loop=1  
--- Mode 3 PASSED: TX(0x55) == RX(0x55) ---
```

SPI - Logic Analyzer

SCLK에 맞춰 MOSI신호로 데이터 순차적으로 전송

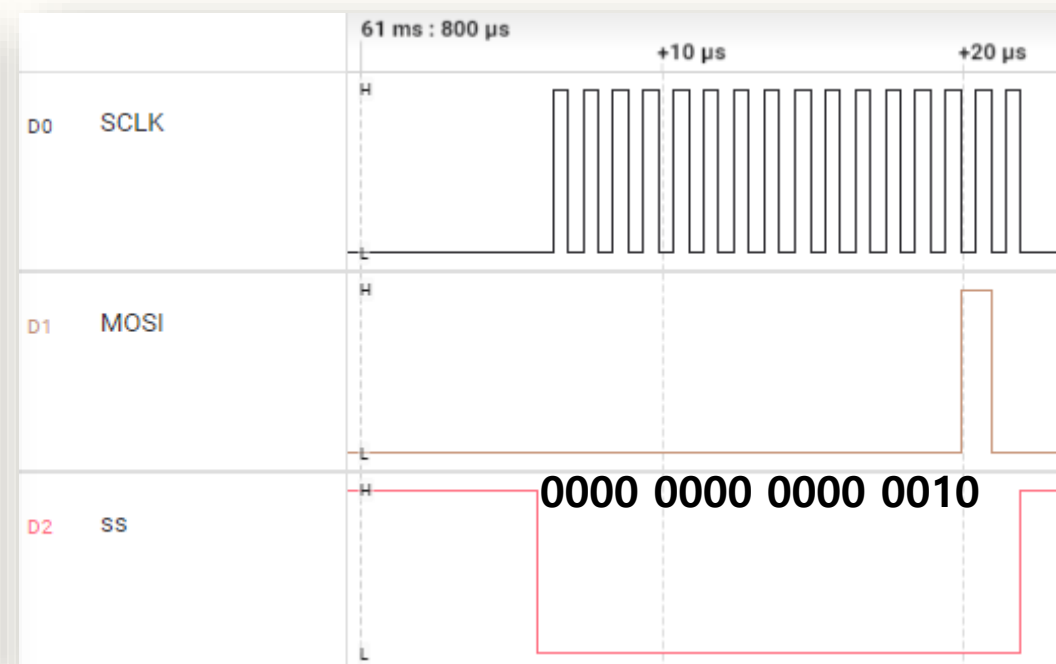
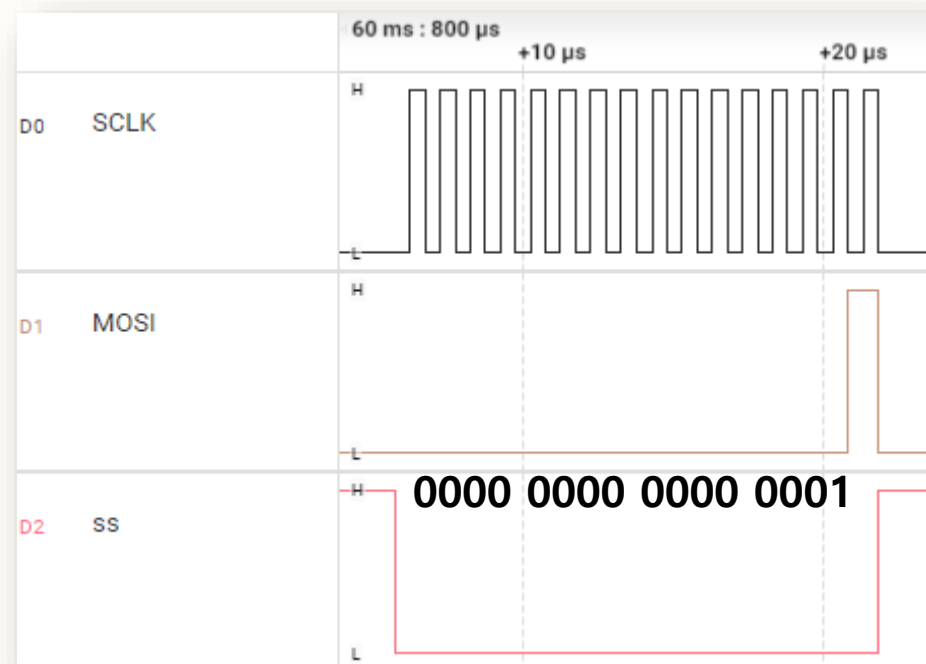
Up_Counter Run



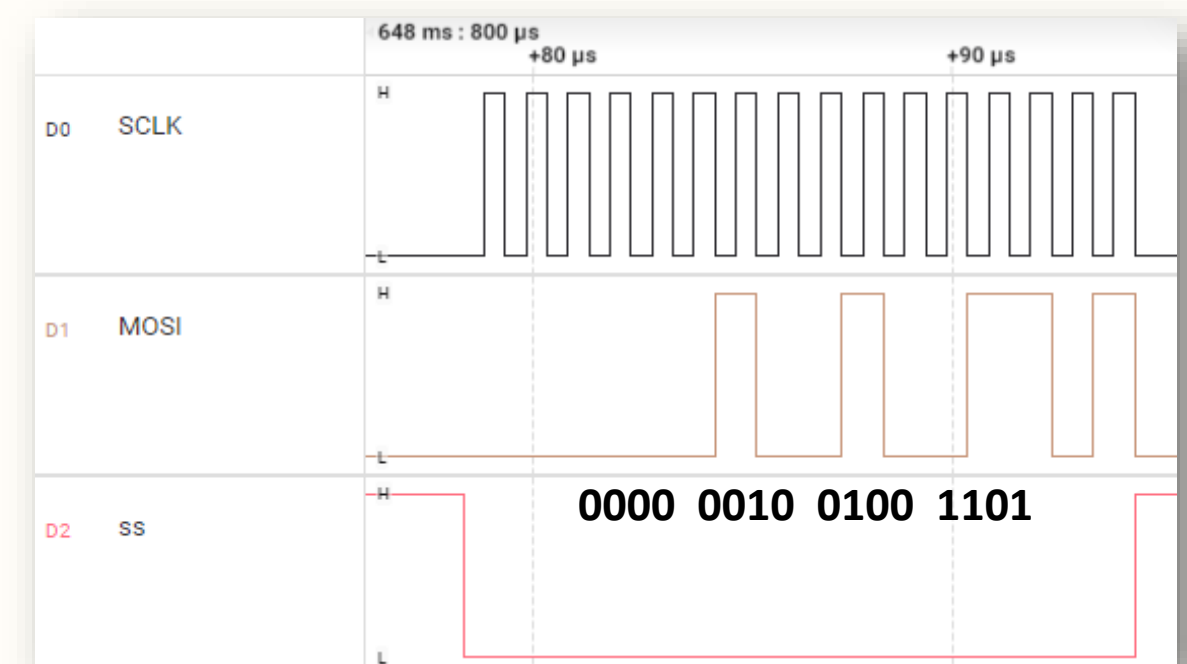
Up_Counter : 1

Up_Counter : 2

Up_Counter : 589



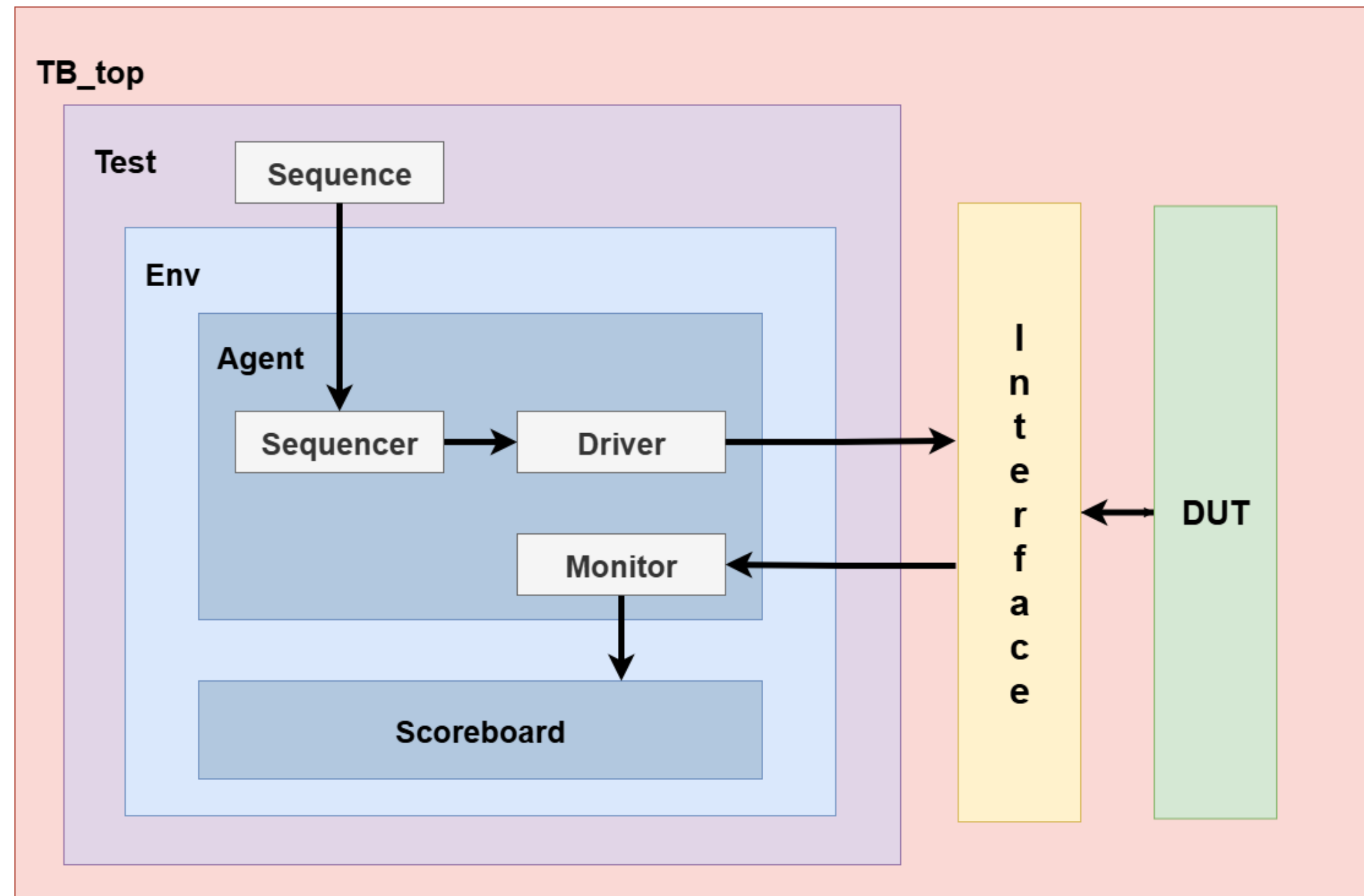
...



...

SPI Protocol

– UVM



SPI Protocol

- UVM

Sequence Item들을 생성하고, 무작위 값을 부여하며,
Driver로 전송하는 **트랜잭션 흐름**을 정의

```
class spi_sequence extends uvm_sequence #(spi_seq_item);
```

```
for (int i = 0; i < 10; i++) begin
    start_item(spi_item);

    spi_item.randomize();

    `uvm_info("SEQ", $sformatf(
        "spi item to driver write:%0d, addr_reg:%0d, num_bytes:%0d,",
        spi_item.write,
        spi_item.addr_reg,
        spi_item.num_bytes
    ), UVM_NONE)

    finish_item(spi_item);
end
```

Sequence

Sequencer

```
class spi_seq_item extends uvm_sequence_item;
    rand bit write;
    rand bit [1:0] addr_reg;
    rand logic [7:0] data_bytes[4];
    rand int unsigned num_bytes;
    logic [7:0] tx_bytes;
    logic [7:0] rx_bytes;

    constraint cnt_c {num_bytes inside {[1:4]}};

    function new(string name = "spi_seq_item");
        super.new(name);
    endfunction //new()

    `uvm_object_utils_begin(spi_seq_item)
        `uvm_field_int(write, UVM_DEFAULT)
        `uvm_field_int(addr_reg, UVM_DEFAULT)
        `uvm_field_int(num_bytes, UVM_DEFAULT)
        `uvm_field_int(rx_bytes, UVM_DEFAULT)
        `uvm_field_int(tx_bytes, UVM_DEFAULT)
    `uvm_object_utils_end

endclass //spi_seq_item extends uvm_sequence_item
```

DUT로 전송될 또는 DUT로부터 수신될
트랜잭션(Transaction) 데이터를 캡슐화

Sequencer로부터 Sequence Item을 받아
DUT의 Interface 신호에 맞게 **실제 연결**

```
class spi_driver extends uvm_driver #(spi_seq_item);
    `uvm_component_utils(spi_driver)

    virtual task run_phase(uvm_phase phase);
        forever begin
            seq_item_port.get_next_item(spi_item);
            s_if.addr = 1;

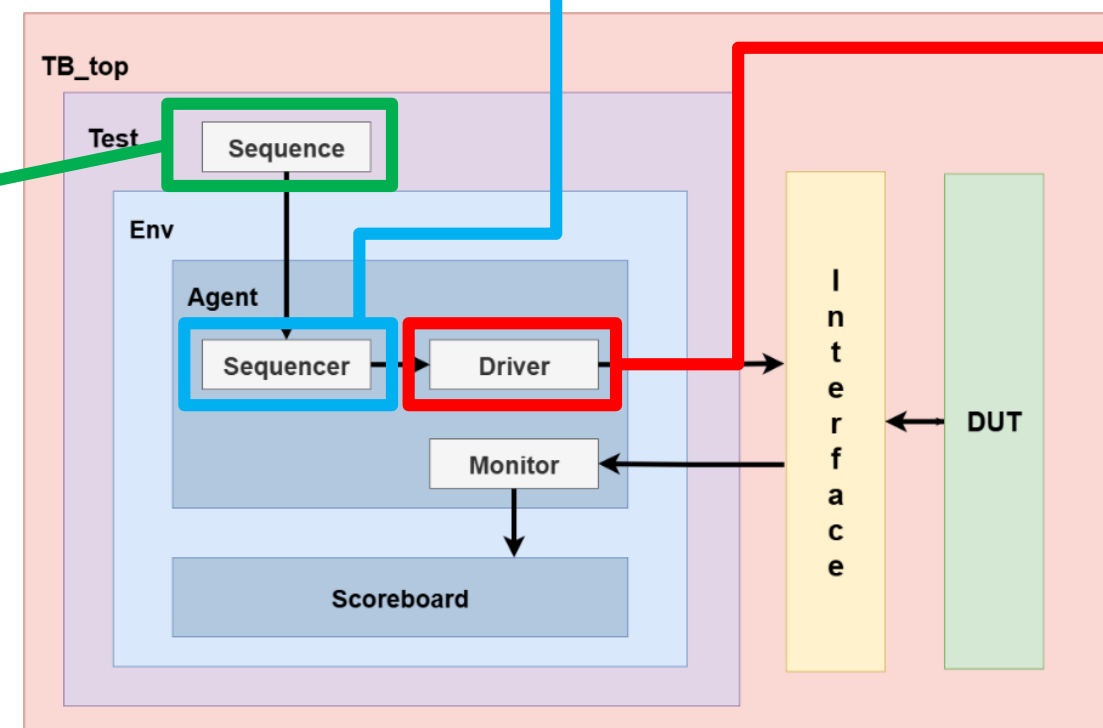
            cmd = {spi_item.write, 5'b0, spi_item.addr_reg};
            send_byte(cmd);
            #50;

            for (int i = 0; i < spi_item.num_bytes; i++) begin
                #50;
                send_byte(spi_item.data_bytes[i]);
            end
            s_if.addr = 0;
            #10;
            seq_item_port.item_done();
        end
    endtask

    task send_byte(byte b);
        s_if.tx_data = b;
        s_if.cpol = 0;
        s_if.cpha = 0;
        s_if.start = 1;
        s_if.num_bytes = spi_item.num_bytes;
        `uvm_info("DRV",
            $sformatf("Drive DUT write:%0d, addr_reg:%0d, num_bytes:%0d",
                spi_item.write, spi_item.addr_reg,
                spi_item.num_bytes), UVM_NONE)

        @(posedge s_if.clk);
        @(posedge s_if.clk);
        @(posedge s_if.clk);
        @(posedge s_if.clk);
        s_if.start = 0;
        @(posedge s_if.done);
        #100;
    endtask //
```

Driver



SPI Protocol – UVM

Scoreboard

Monitor로부터 캡처된 트랜잭션과
Golden Reference Model의 결과를 비교하여
DUT의 동작이 올바른지 확인

DUT의 Interface 신호를 감시하여
트랜잭션을 캡처하고, Scoreboard로 전송

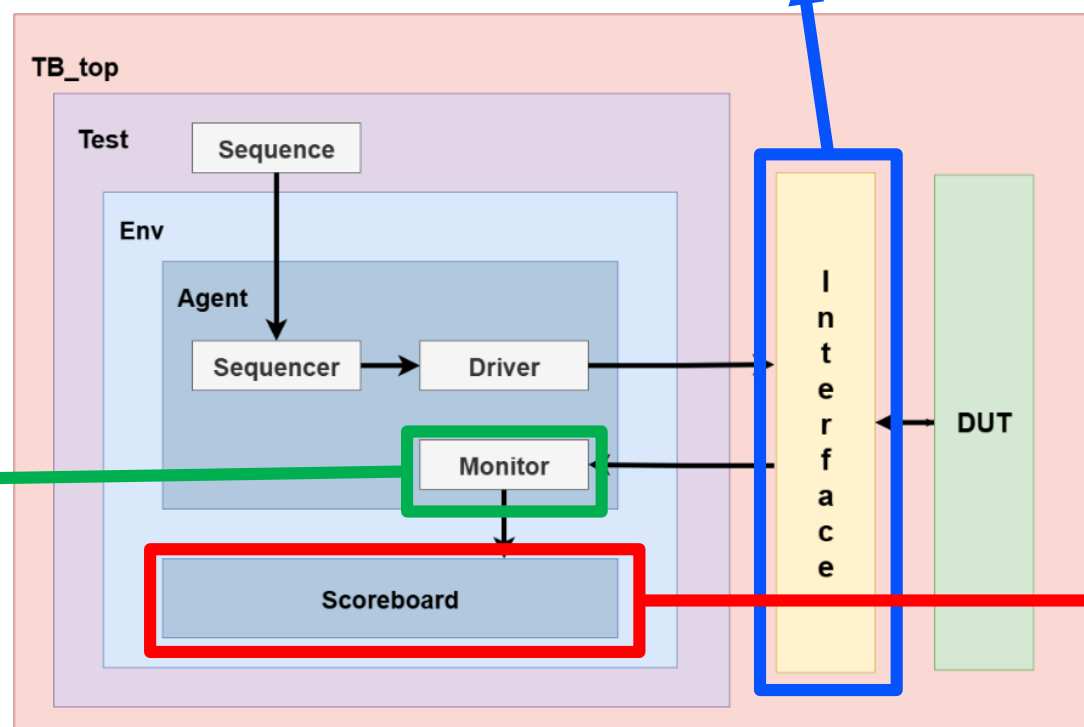
```
class spi_monitor extends uvm_monitor;
  `uvm_component_utils(spi_monitor)

  virtual task run_phase(uvm_phase phase);
  forever begin
    byte cnt = 0;
    @(posedge s_if.done);
    @(posedge s_if.clk);
    @(posedge s_if.clk);
    @(posedge s_if.clk);
    spi_item.write = s_if.tx_data[7];
    spi_item.addr_reg = s_if.tx_data[1:0];
    spi_item.num_bytes = s_if.num_bytes;
    spi_item.tx_bytes = s_if.tx_data;
    spi_item.rx_bytes = s_if.rx_data;
    `uvm_info("MON", $sformatf(
      "sampled write:%0d, addr_reg:%0d, num_bytes:%0d, tx_byte : %d, rx_byte = %d",
      spi_item.write,
      spi_item.addr_reg,
      spi_item.num_bytes,
      spi_item.tx_bytes,
      spi_item.rx_bytes
    ), UVM_NONE)
    send.write(spi_item); // send to scoreboard
  end
endtask
```

Monitor

```
interface spi_if;
  logic clk;
  logic reset;
  logic cpol;
  logic cpha;
  logic start;
  logic addr;
  logic [7:0] tx_data;
  logic [7:0] rx_data;
  logic done;
  logic ready;
  int unsigned num_bytes;
endinterface //spi_if
```

Interface



```
class spi_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(spi_scoreboard)

  if (addr_after == 1 && WR == 1) begin
    golden_reg[slv_addr] = spi_item.tx_bytes;
    `uvm_info("SCO", $sformatf("Write success addr : %d, data : %d",
      slv_addr, spi_item.tx_bytes), UVM_LOW)
    slv_addr++;
    count_number--;
  end

  if (addr_after == 1 && WR == 0) begin
    if (spi_item.rx_bytes == golden_reg[slv_addr]) begin
      `uvm_info("SCO", $sformatf(
        "*** TEST PASSED *** , rx_Data : %0d, data : %0d, addr : %0d",
        spi_item.rx_bytes,
        golden_reg[slv_addr],
        slv_addr
      ), UVM_NONE);
      slv_addr++;
      count_number--;
    end else begin
      `uvm_error("SCO", $sformatf(
        "*** TEST FAIL *** , rx_Data : %0d, data : %0d, addr : %0d",
        spi_item.rx_bytes,
        golden_reg[slv_addr],
        slv_addr
      ))
      slv_addr++;
      count_number--;
    end
  end

  if (addr_after == 0) begin
    WR = spi_item.tx_bytes[7];
    addr_after = 1;
    slv_addr = spi_item.tx_bytes[1:0];
    count_number = spi_item.num_bytes;
    `uvm_info("SCO", $sformatf(
      "*** data_start *** , rx_Data : %0d, data : %0d, addr : %0d",
      spi_item.rx_bytes,
      golden_reg[slv_addr],
      slv_addr
    ), UVM_NONE);
  end

  if (count_number == 0) begin
    addr_after = 0;
  end
end
```

SPI Protocol – UVM

Driver, Sequencer, Monitor를 묶어 만든
하나의 재사용 가능한 모듈

Agent

```
class spi_agent extends uvm_agent;
  `uvm_component_utils(spi_agent)
  function new(string name = "AGENT", uvm_component parent);
    super.new(name, parent);
  endfunction //new()

  spi_monitor spi_mon;
  spi_driver spi_drv;
  uvm_sequencer #(spi_seq_item) spi_sqr;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    spi_mon = spi_monitor::type_id::create("MON", this);
    spi_drv = spi_driver::type_id::create("DRV", this);
    spi_sqr = uvm_sequencer #(spi_seq_item)::type_id::create("SQR", this);
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    spi_drv.seq_item_port.connect(spi_sqr.seq_item_export);
  endfunction
endclass //spi_agent extends uvm_agent
```

```
class spi_environment extends uvm_env;
  `uvm_component_utils(spi_environment)

  function new(string name = "ENV", uvm_component parent);
    super.new(name, parent);
  endfunction //new()

  spi_scoreboard spi_sco;
  spi_agent spi_agt;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    spi_sco = spi_scoreboard::type_id::create("SCO", this);
    spi_agt = spi_agent::type_id::create("AGT", this);
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    spi_agt.spi_mon.send.connect(
      spi_sco.recv);
  endfunction
endclass //spi_environment extends uvm_env
```

Environment

Agent와 Scoreboard를 포함
전체 검증 환경의 최상위 컨테이너 역할

Test

```
class test extends uvm_test;
  `uvm_component_utils(test)

  function new(string name = "TEST", uvm_component parent);
    super.new(name, parent);
  endfunction //new()

  spi_sequence spi_seq;
  spi_envirment spi_env;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    spi_seq = spi_sequence::type_id::create("SEQ", this);
    spi_env = spi_envirment::type_id::create("ENV", this);
  endfunction

  virtual task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    spi_seq.start(spi_env.spi_agt.spi_sqr);
    phase.drop_objection(this);
  endtask //
endclass //test extends uvm_test
```

검증 환경을 구축
Sequence를 실행하여 테스트를 시작

```
module tb_spi ();
  spi_if s_if ();

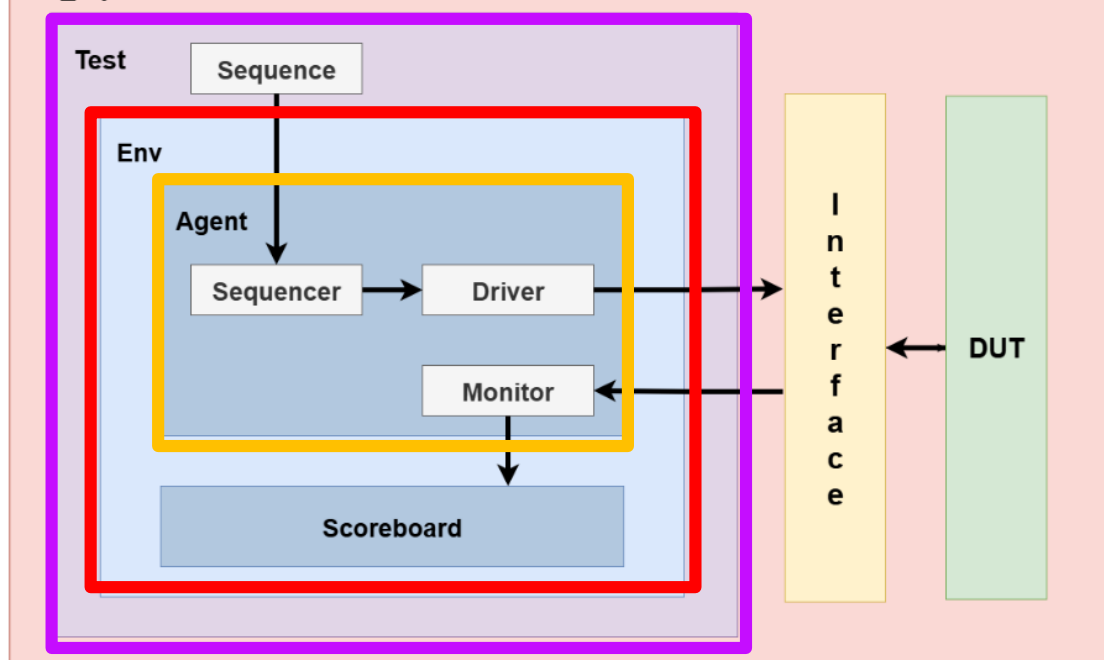
  SPI dut (
    .clk(s_if.clk),
    .reset(s_if.reset),
    .cpol(s_if.cpol),
    .cpha(s_if.cpha),
    .start(s_if.start),
    .addr(s_if.addr),
    .tx_data(s_if.tx_data),
    .rx_data(s_if.rx_data),
    .done(s_if.done),
    .ready(s_if.ready)
  );

  always #5 s_if.clk = ~s_if.clk;
  initial begin
    $fsdbDumpvars(
      0); // verdi tool을 사용하기 위해 testbench에 모든 정보 수집
    $fsdbDumpfile("wave.fsdb");
    s_if.clk = 0;
    s_if.reset = 1;
    uvm_config_db #(virtual spi_if)::set(null, "*", "s_if", s_if);

    run_test();
  end

  initial begin
    #5;
    s_if.reset = 1'b0;
  end
endmodule
```

TB_top



SPI Protocol

– UVM

```
** Report counts by severity
UVM_INFO : 1420
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[DRV] 329
[MON] 329
[RNTST] 1
[SCO] 658
[SEQ] 100
[TEST_DONE] 1
[UVM/RELNOTES] 1
[UVM/REPORT/CATCHER] 1
```

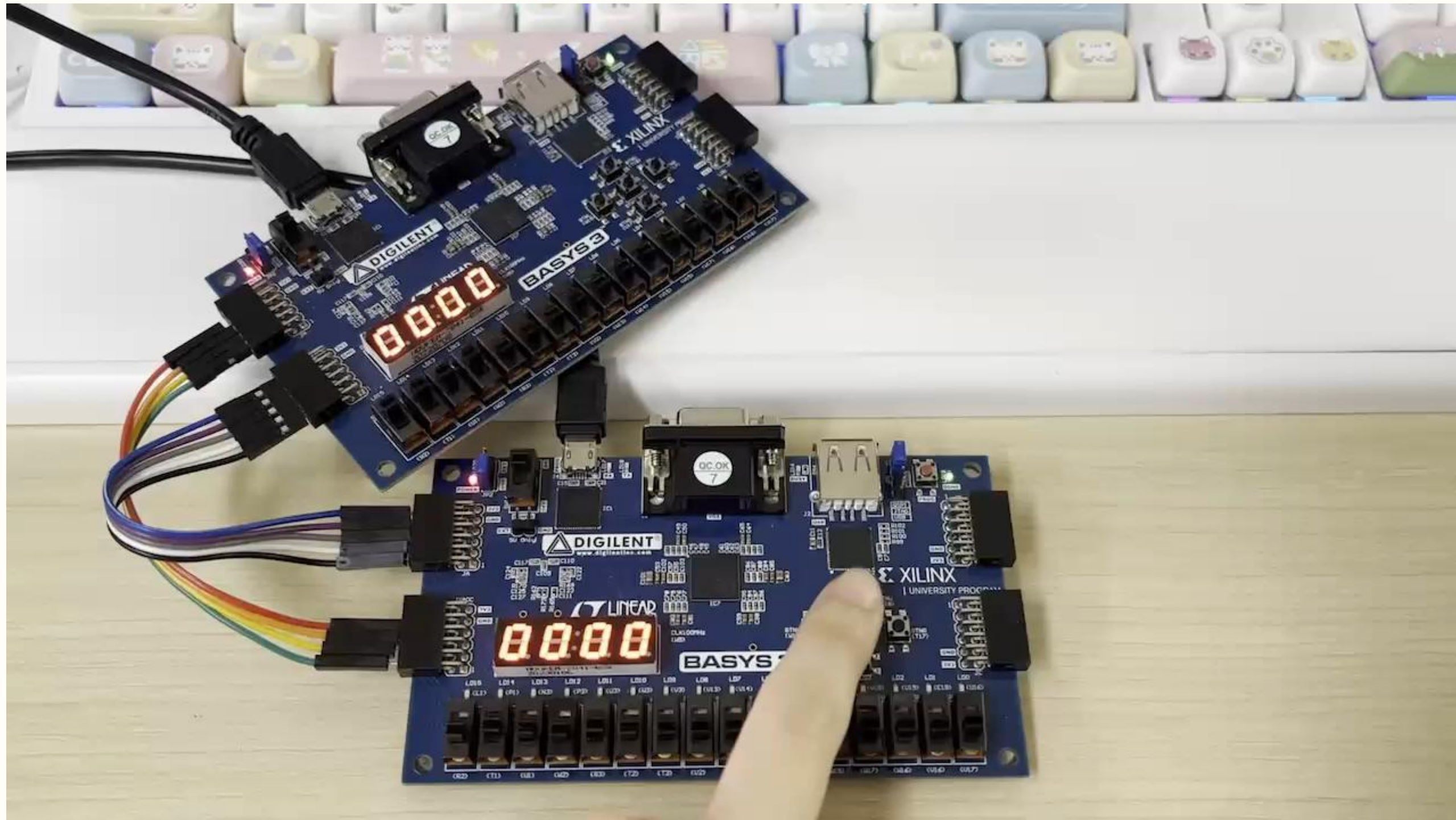
Write

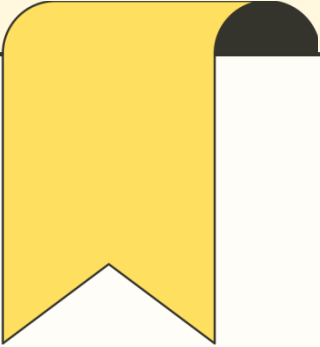
```
uvm_test_top.ENV.SCO [SCO] *** data_start *** , rx_Data : 0, data : 131, addr : 2
uvm_test_top.ENV.AGT.DRV [DRV] Drive DUT write:1, addr_reg:2, num_bytes:3
uvm_test_top.ENV.AGT.MON [MON] sampled write:0, addr_reg:3, num_bytes:3, tx_byte : 75, rx_byte = 0
uvm_test_top.ENV.SCO [SCO] sampled write:0, addr_reg:3, num_bytes:3, tx_byte : 75, rx_byte = 0
uvm_test_top.ENV.SCO [SCO] Write success addr : 2, data : 75
```

Read

```
uvm_test_top.ENV.SCO [SCO] *** data_start *** , rx_Data : 0, data : 75, addr : 2
uvm_test_top.ENV.AGT.DRV [DRV] Drive DUT write:0, addr_reg:2, num_bytes:1
uvm_test_top.ENV.AGT.MON [MON] sampled write:1, addr_reg:2, num_bytes:1, tx_byte : 242, rx_byte = 75
uvm_test_top.ENV.SCO [SCO] sampled write:1, addr_reg:2, num_bytes:1, tx_byte : 242, rx_byte = 75
uvm_test_top.ENV.SCO [SCO] *** TEST PASSED *** , rx_Data : 75, data : 75, addr : 2
```

A yellow bookmark icon with a black outline, positioned at the top right of the page.



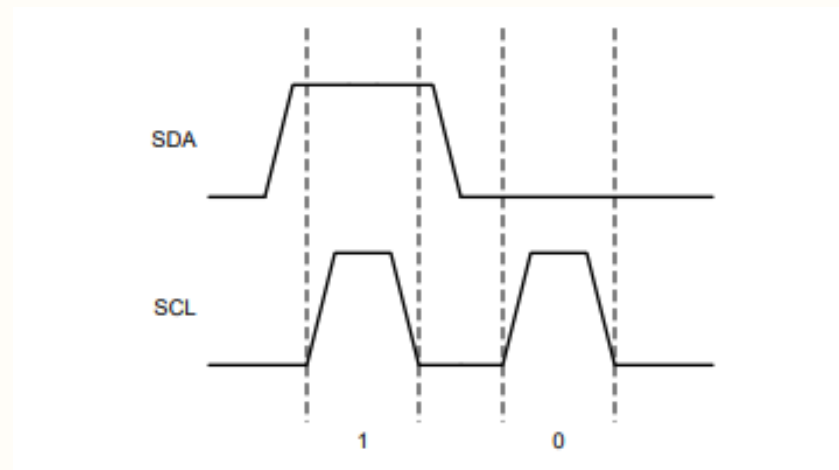
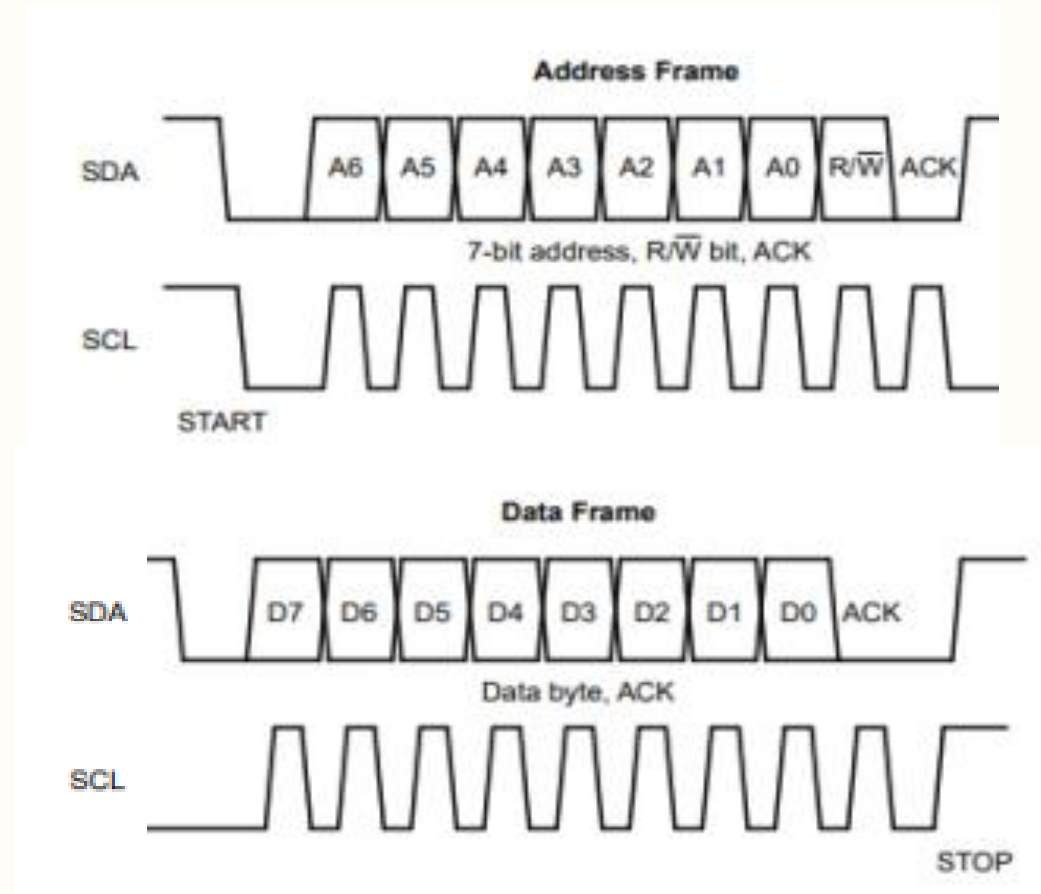


I2C

I2C

I2C : Inter-Integrated Circuit

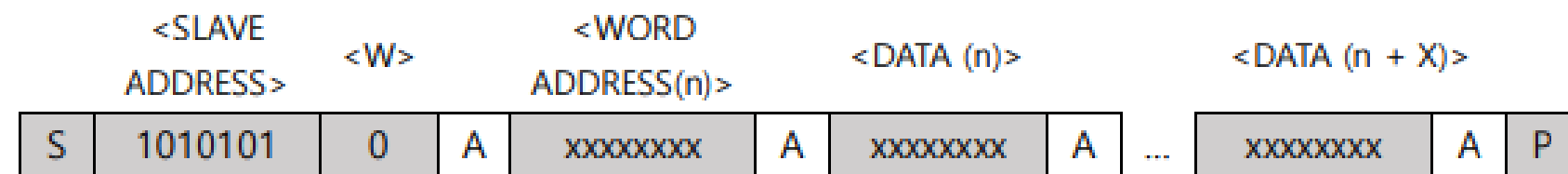
: SDA, SCL만을 사용하여 통신하는 직렬 통신 프로토콜



- **2-wire interface** : SDA (데이터)와 SCL (클럭) 두 선만 사용
- **반이중 통신 (Half Duplex)**: 데이터를 한 번에 **한 방향**으로만 통신이 이루어짐
- Slave 주소 지정 : **7비트 주소로 통신 대상을 선택**
- **ACK/NACK 응답** : 데이터 수신 성공/실패 여부를 피드백하여 신뢰성 확보
- **오픈-드레인 구조** : 풀업 저항이 필수이며, 여러 장치 연결 및 클럭 스트레칭 지원

I2C Protocol

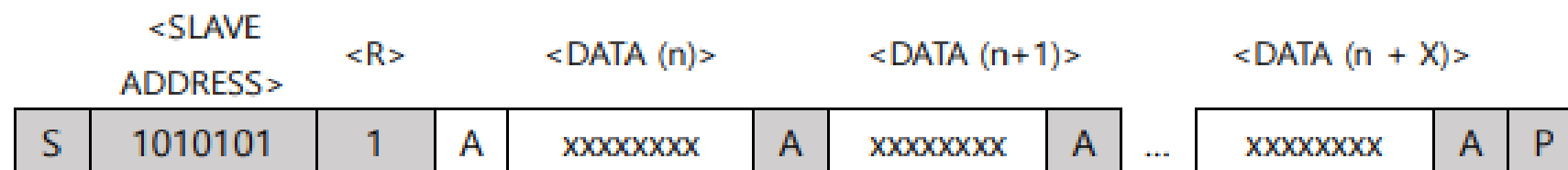
Write



S START
A ACKNOWLEDGE (ACK)
P STOP
W WRITE BIT ADDRESS

☐ SLAVE TO MASTER ☒ MASTER TO SLAVE

Read



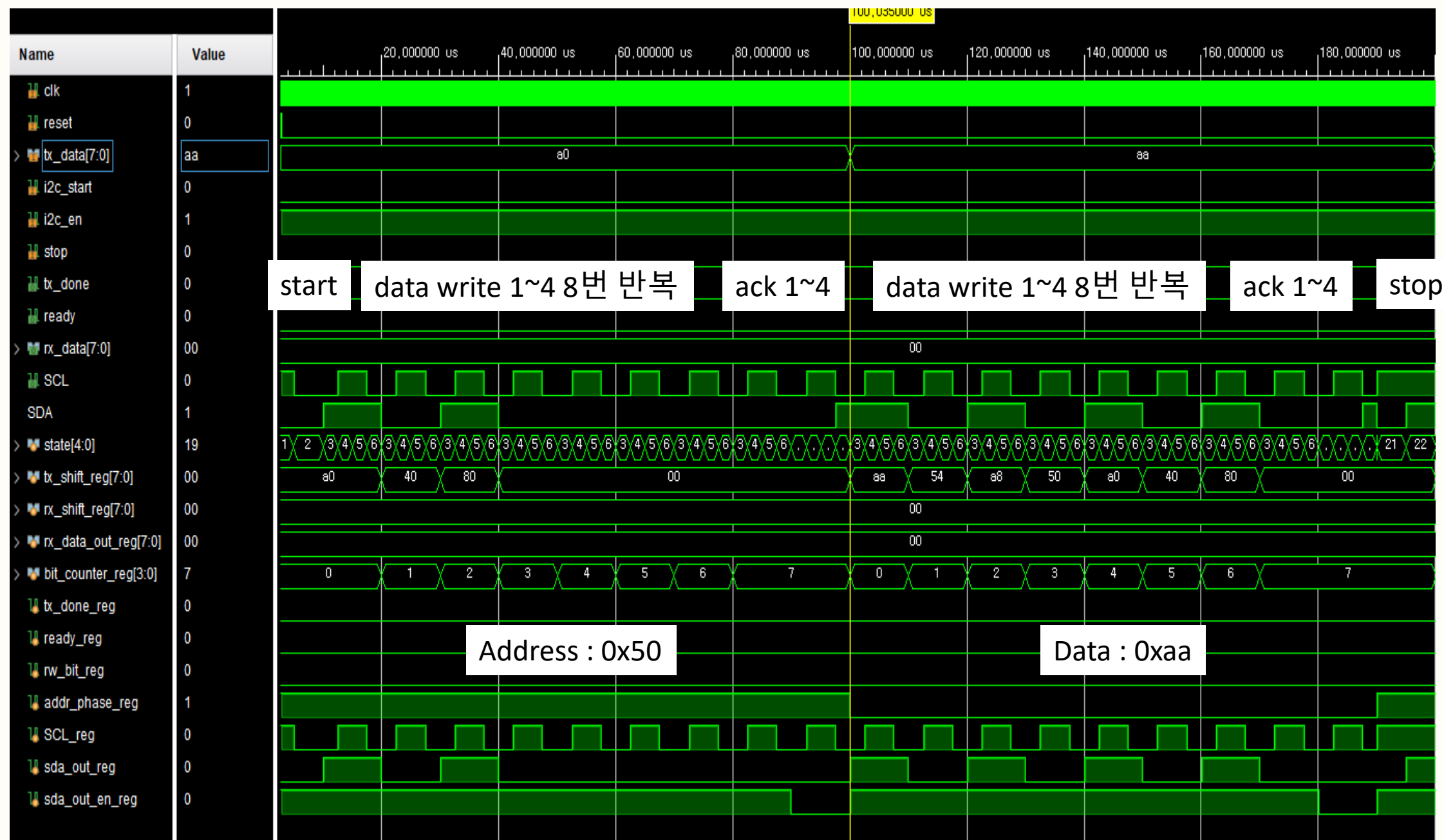
S START
A ACKNOWLEDGE (ACK)
P STOP
W WRITE BIT ADDRESS
 $\bar{A}$ NOT ACKNOWLEDGE

☐ SLAVE TO MASTER ☒ MASTER TO SLAVE

- **S (Start)** : 통신 시작 신호
- **Address** : 통신할 **Slave** 주소 지정
- **R/W bit**: 읽기(1) 또는 쓰기(0) 결정
- **Data** : 8비트 단위의 실제 정보 교환
- **P (Stop)**: 통신 종료 신호

I2C Protocol

– Master : Write Simulation



```
[25000] Master is READY.
[25000] --- Test 1: I2C Write (Addr 0x50, Data 0xaa) ---
[25000] State: 0 | SCL: 1 | SDA: 1 | Ready: 1 | Done: 0 | TX_Data: a0 | RX_Data: 00
[35000] State: 1 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[45000] State: 1 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[5035000] State: 2 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[5045000] State: 2 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[10035000] State: 3 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[10045000] State: 3 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[12535000] State: 4 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[12545000] State: 4 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[15035000] State: 5 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[17535000] State: 6 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[17545000] State: 6 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
...
[90035000] State: 11 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[92535000] State: 12 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[92545000] State: 12 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[95035000] State: 13 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[97535000] State: 14 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[97545000] State: 14 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: a0 | RX_Data: 00
[100035000] Master finished Address, providing Data Byte 0xaa.
[100045000] State: 3 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[102545000] State: 4 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[102555000] State: 4 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[105045000] State: 5 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[107545000] State: 6 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[107555000] State: 6 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
...
[180045000] State: 11 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[182545000] State: 12 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[182555000] State: 12 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[185045000] State: 13 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[187545000] State: 14 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[187555000] State: 14 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[190045000] State: 19 | SCL: 0 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[190055000] State: 21 | SCL: 0 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[190065000] State: 21 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[195055000] State: 22 | SCL: 1 | SDA: 0 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[195065000] State: 22 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 0 | TX_Data: aa | RX_Data: 00
[200055000] State: 0 | SCL: 1 | SDA: 1 | Ready: 0 | Done: 1 | TX_Data: aa | RX_Data: 00
[200065000] --- Test 1: Write complete. ---
```

start

Data
Write
1~4
: address

ack
1~4

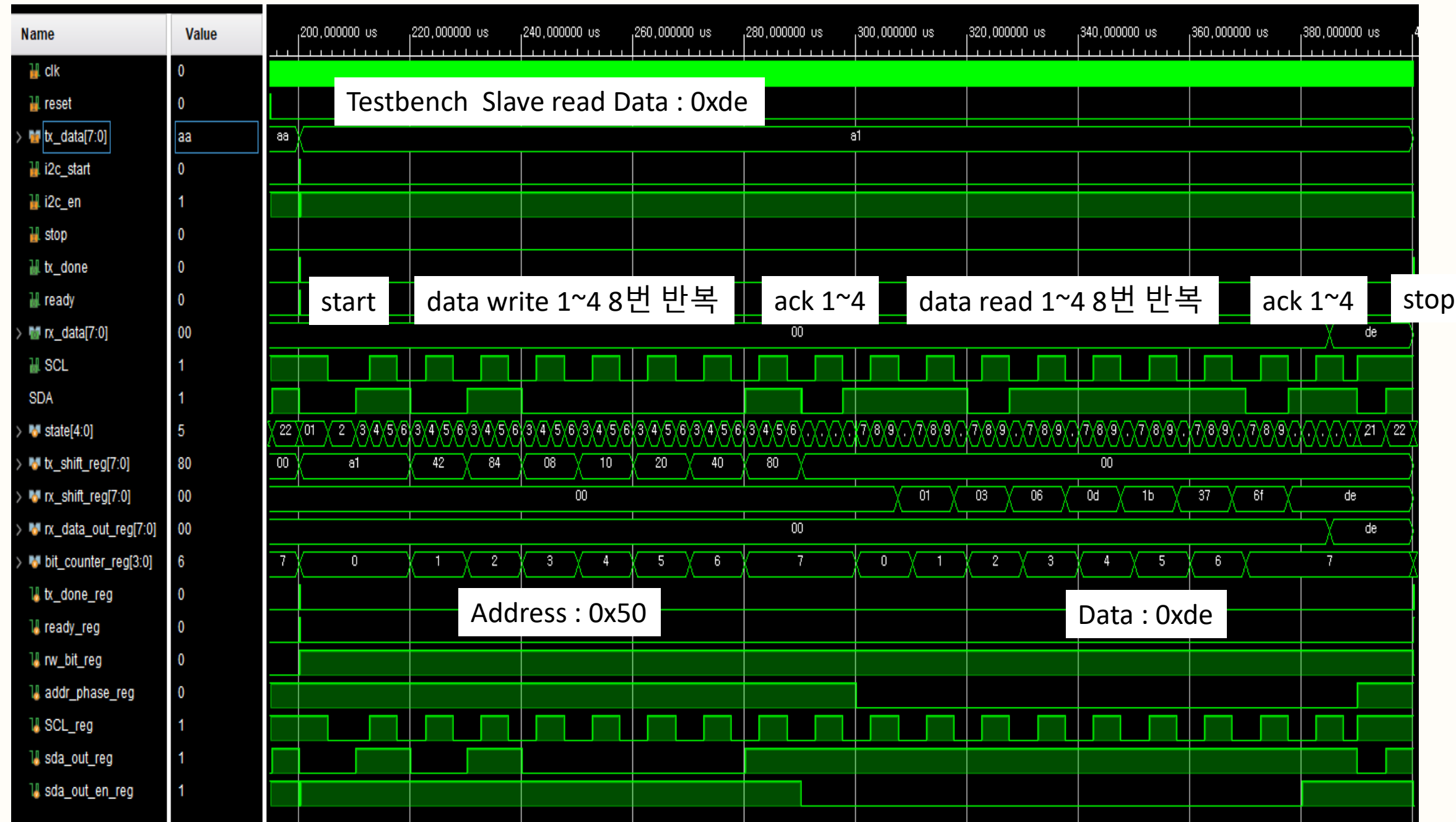
Data
Write
1~4
: data

ack
1~4

stop

I2C Protocol

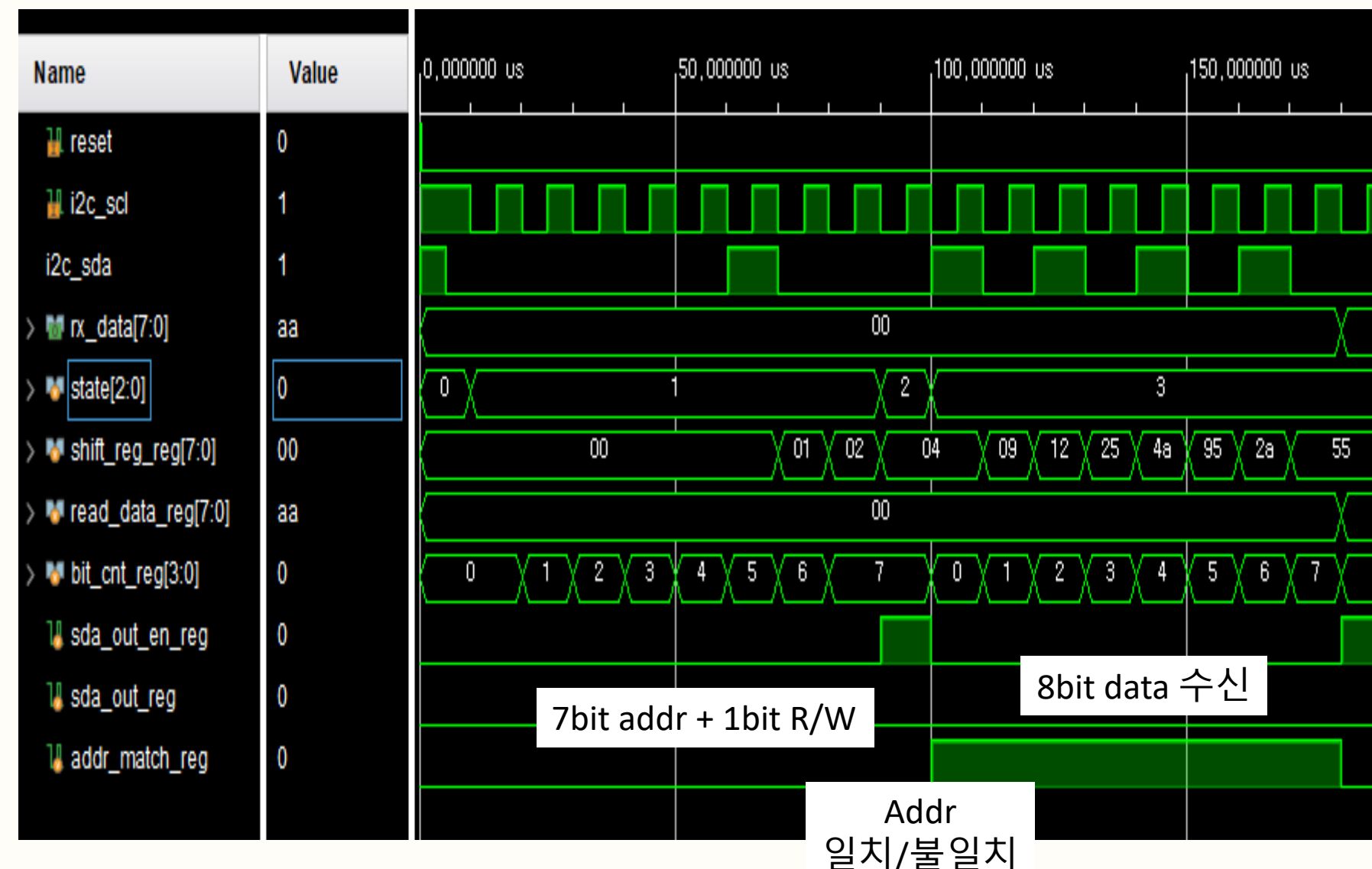
– Master : Read Simulation



[200165000] --- Test 2: I2C Read (Addr 0x50) ---	
[200165000] Master is READY.	
[200165000] State: 0 SCL: 1 SDA: 1 Ready: 1 Done: 0 TX_Data: a1 RX_Data: 00	start
[200175000] State: 1 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[200185000] State: 1 SCL: 1 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[205175000] State: 2 SCL: 1 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[205185000] State: 2 SCL: 0 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	Data Write 1~4 : address
[210175000] State: 3 SCL: 0 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[210185000] State: 3 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[212675000] State: 4 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[212685000] State: 4 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[215175000] State: 5 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[217675000] State: 6 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[217685000] State: 6 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
...	
[290175000] State: 11 SCL: 0 SDA: x Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	ack 1~4
[290185000] State: 11 SCL: 0 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[292675000] State: 12 SCL: 0 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[292685000] State: 12 SCL: 1 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[295175000] State: 13 SCL: 1 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	Data Read 1~4 : data
[297675000] State: 14 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[297685000] State: 14 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[300175000] State: 15 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[300185000] State: 7 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	ack 1~4
[302685000] State: 8 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[302695000] State: 8 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[305185000] State: 9 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[307685000] State: 10 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	stop
[307695000] State: 10 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
...	
[380185000] State: 15 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[382685000] State: 16 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	ack 1~4
[382695000] State: 16 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[385185000] State: 17 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: 00	
[385195000] State: 17 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[387685000] State: 18 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	stop
[387695000] State: 18 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[390185000] State: 20 SCL: 0 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[390195000] State: 21 SCL: 0 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[390205000] State: 21 SCL: 1 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[395195000] State: 22 SCL: 1 SDA: 0 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[395205000] State: 22 SCL: 1 SDA: 1 Ready: 0 Done: 0 TX_Data: a1 RX_Data: de	
[400195000] State: 0 SCL: 1 SDA: 1 Ready: 0 Done: 1 TX_Data: a1 RX_Data: de	
[400205000] --- Test 2: Read PASSED. Got 0xde ---	
[400205000] State: 0 SCL: 1 SDA: 1 Ready: 1 Done: 0 TX_Data: a1 RX_Data: de	
[400305000] --- All tests complete. ---	

I2C Protocol

– Slave : Write Simulation



[1000000] --- Test 1: I2C Write (Data 0xaa) ---

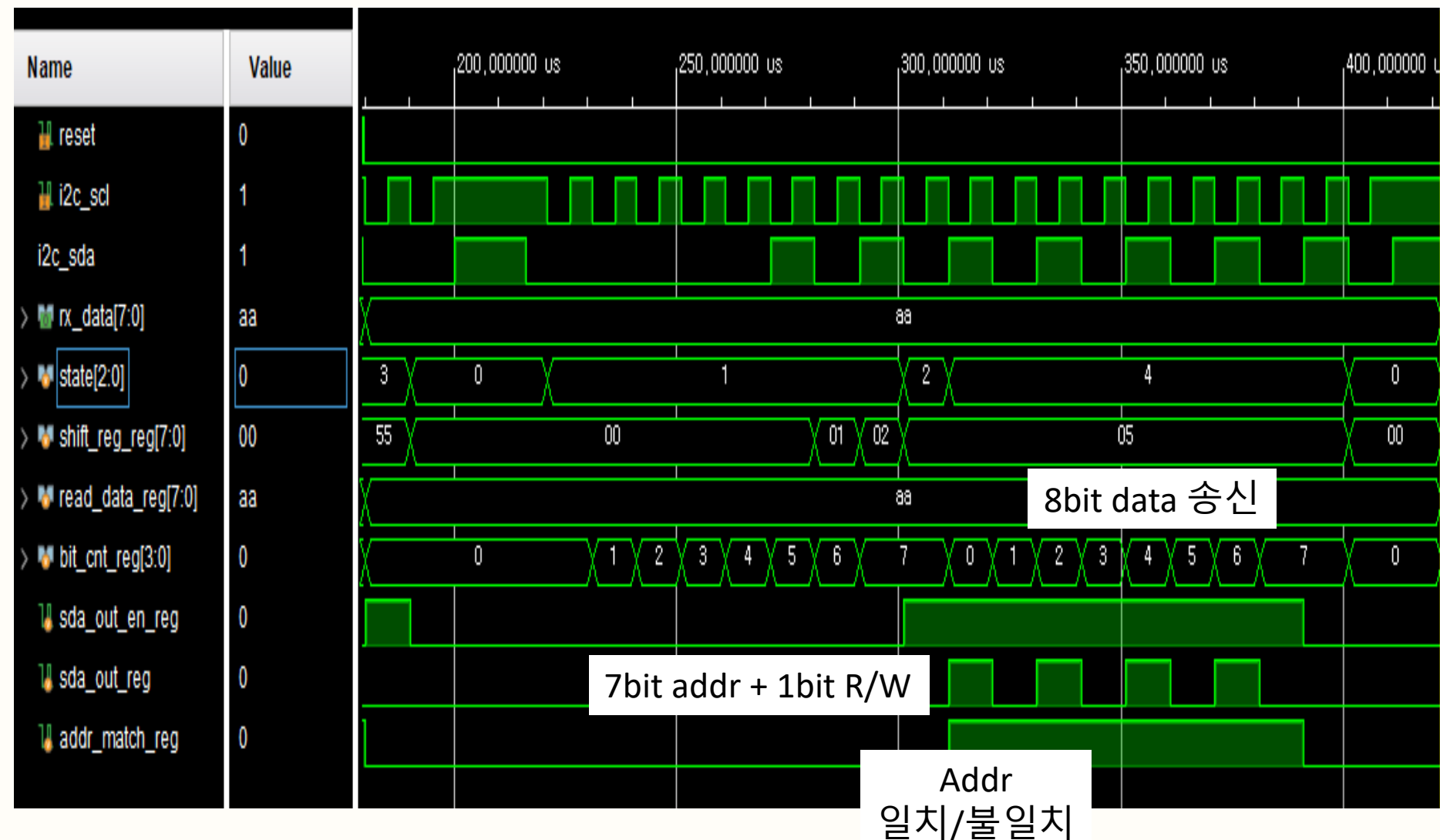
[5100000] SCL: 1 | SDA: 0 | Slave State: 0 | rx_data: 00
 [10100000] SCL: 0 | SDA: 0 | Slave State: 1 | rx_data: 00
 [15100000] SCL: 1 | SDA: 0 | Slave State: 1 | rx_data: 00
 [20100000] SCL: 0 | SDA: 0 | Slave State: 1 | rx_data: 00

[90100000] SCL: 0 | SDA: 0 | Slave State: 2 | rx_data: 00
 [95100000] SCL: 1 | SDA: 0 | Slave State: 2 | rx_data: 00
 [100100000] SCL: 0 | SDA: 1 | Slave State: 3 | rx_data: 00
 [105100000] SCL: 1 | SDA: 1 | Slave State: 3 | rx_data: 00

[201100000] Test 1: Slave ACK address and data correctly.
 [201100000] --- Test 1: Write PASS. 'rx_data' output is 0xaa ---

I2C Protocol

– Slave : Read Simulation

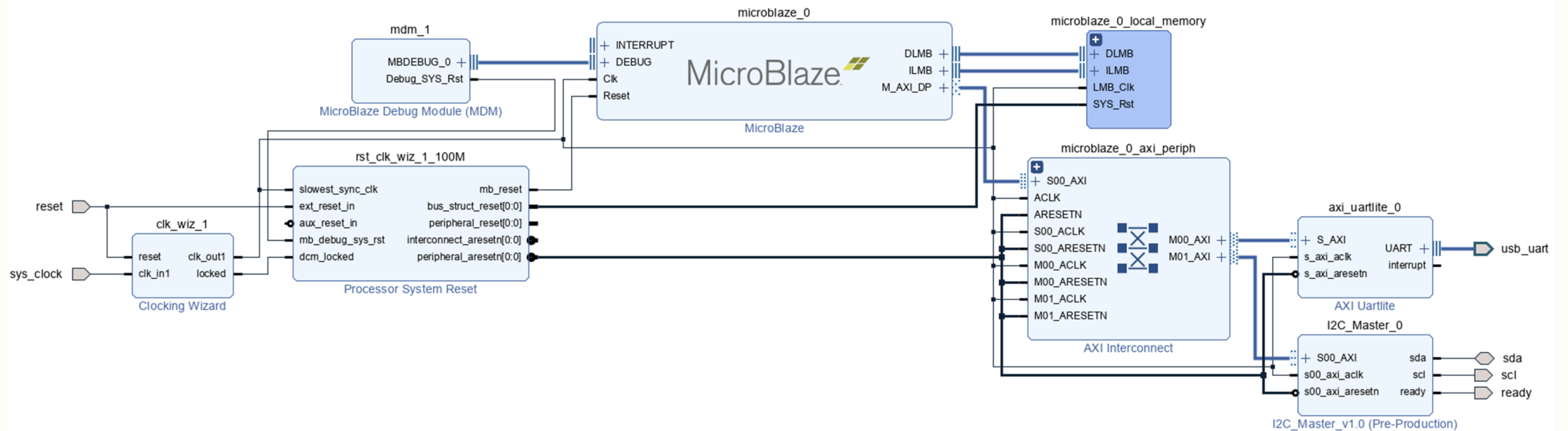


```
[211100000] --- Test 2: I2C Read (Expecting 0xaa) ---
[216100000] SCL: 1 | SDA: 0 | Slave State: 0 | rx_data: aa
[221100000] SCL: 0 | SDA: 0 | Slave State: 1 | rx_data: aa
[226100000] SCL: 1 | SDA: 0 | Slave State: 1 | rx_data: aa
[231100000] SCL: 0 | SDA: 0 | Slave State: 1 | rx_data: aa
```

```
[301100000] SCL: 0 | SDA: 0 | Slave State: 2 | rx_data: aa
[306100000] SCL: 1 | SDA: 0 | Slave State: 2 | rx_data: aa
[311100000] SCL: 0 | SDA: 1 | Slave State: 4 | rx_data: aa
[316100000] SCL: 1 | SDA: 1 | Slave State: 4 | rx_data: aa
```

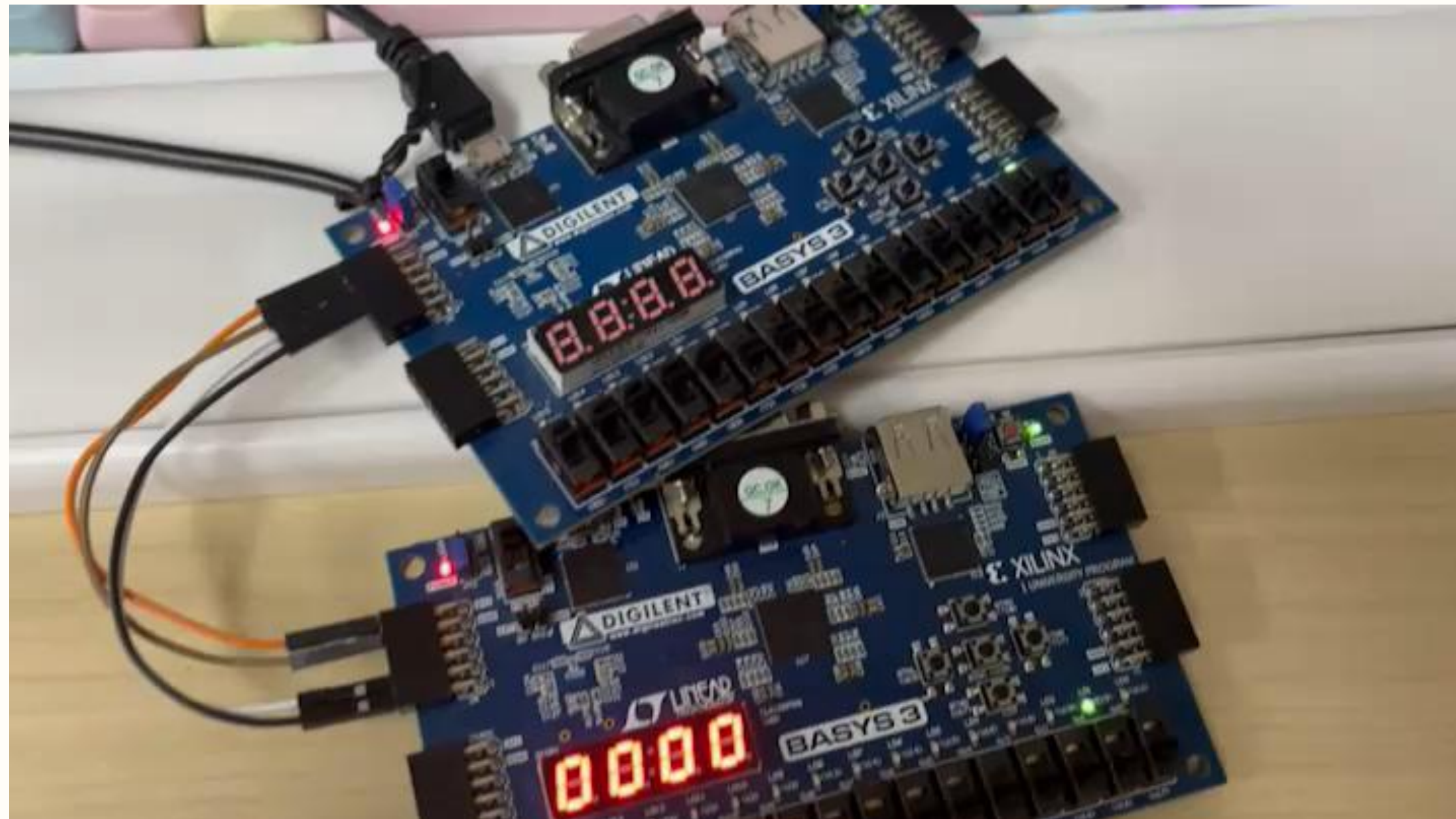
```
[412100000] Test 2: Slave ACK read address correctly.
[412100000] --- Test 2: Read PASS. Got 0xaa ---
```

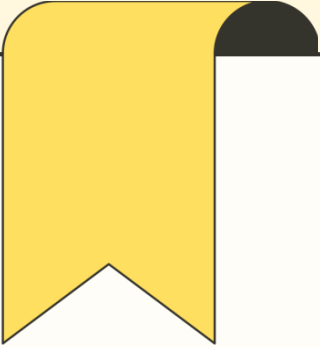
I2C Block Design



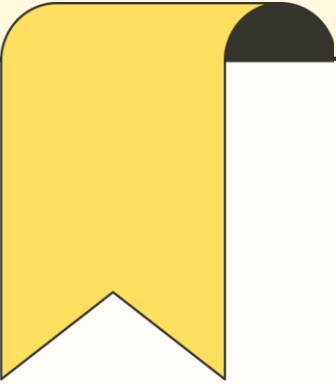
I2C - Demo

Master에서 신호를 주면 LED0을 통해 Slave Ready를 알림





Discussion

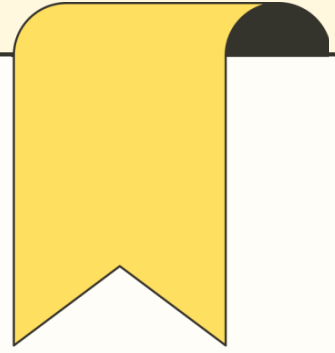


고찰

- **프로토콜 기반 설계 능력 향상:** AXI, SPI, I2C와 같은 실제 산업에서 사용되는 표준 프로토콜을 직접 설계하고 시뮬레이션함으로써, 단순히 이론을 습득하는 것을 넘어 실제 RTL 코드 작성 능력과 프로토콜의 복잡한 타이밍을 제어하는 능력을 크게 향상시킬 수 있었습니다.
- **UVM 검증 환경 구축의 실습:** SystemVerilog 기반의 UVM Testbench 구조를 처음부터 직접 구축하고 Sequence Item, Sequence, Driver, Monitor, Scoreboard와 같은 컴포넌트들을 연결하는 과정을 통해 재사용 가능한 검증 환경을 설계하는 방법을 체득했습니다. 이는 대규모 SoC 개발 환경에서 필수적인 검증 방법론을 이해하는 데 결정적인 경험이었습니다.

향후 계획: I2C 프로토콜 UVM 검증 환경 구축 및 Coverage 기반 검증 프로세스 적용

- 이번 프로젝트에서 구축한 **SPI UVM Testbench** 경험을 발판 삼아, **I2C 프로토콜**을 위한 완전하고 체계적인 UVM 환경을 설계하고 구현
- 현재 구축된 UVM 환경에 Functional Coverage를 추가하여, 랜덤하게 생성된 테스트 케이스가 프로토콜의 모든 가능한 시나리오를 얼마나 잘 검증했는지 확인



Thank you